Análise Amortizada

Complexidade Amortizada

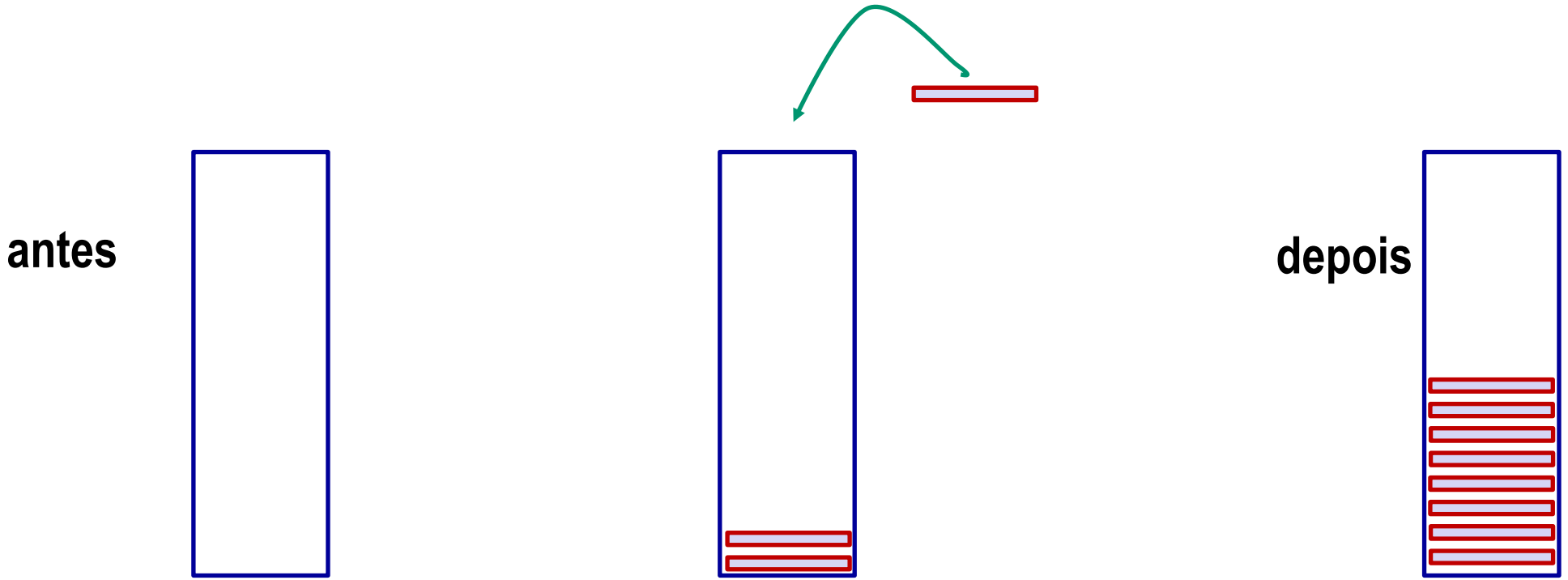
A *Análise Amortizada* calcula o tempo requerido para executar uma **sequência de operações** em uma *estrutura de dados*.

Calcula-se a média do tempo de todas as operações executadas.



Complexidade Amortizada

Consideramos as configurações da estrutura antes e depois de uma sequência de operações. Examina-se cada operação como parte integrante da sequência.



Complexidade Amortizada

Assim, cada operação individual não deverá ser considerada, necessariamente, no seu *pior caso*.

A Análise Amortizada garante "o desempenho médio de cada operação no pior caso".

A Análise Amortizada é diferente da Análise de Caso Médio, uma vez que não existe probabilidade envolvida nos cálculos.

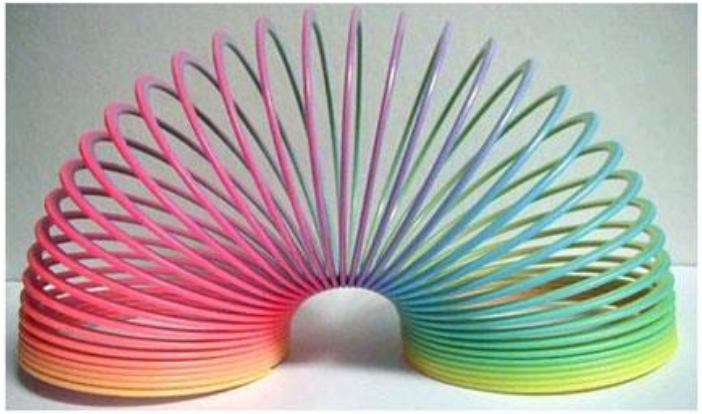
Complexidade Amortizada

Métodos mais utilizados

-Análise Agregada

-Método do Contador

-Método do Potencial



Análise Agregada

Nesse método mostra-se que, para todo n, uma sequência de n operações tem pior caso $T(n)$ no total.

No pior caso, o custo amortizado de cada operação é, portanto, $T(n)/n$.

Análise Agregada

Exemplo: Operações com Pilhas

Lembrando: Empilha $\rightarrow \Theta(1)$

Desempilha $\rightarrow \Theta(1)$

► Considerar o custo de cada operação empilha ou desempilha igual a 1.

O custo total de n operações é $\Theta(n)$



Análise Agregada

Exemplo: Operações com Pilhas

Lembrando: Empilha $\rightarrow \Theta(1)$

Desempilha $\rightarrow \Theta(1)$

► Considerar o custo de cada operação empilha ou desempilha igual a 1.

O custo total de n operações é $\Theta(n)$



Análise Agregada

Exemplo: Operações com Pilhas

Lembrando: Empilha $\rightarrow \Theta(1)$

Desempilha $\rightarrow \Theta(1)$

► Considerar o custo de cada operação empilha ou desempilha igual a 1.

O custo total de n operações é $\Theta(n)$



Análise Agregada

Exemplo: Operações em Pilhas

- *desempilha_mult*(P,*k*) retira *k* elementos da pilha P

```
desempilha_mult(P,k)
   enquanto ((pilha não vazia) e (k ≠ 0)) faça
      desempilha(P)
      k ← k – 1
```

Análise Agregada

Exemplo: Operações em Pilhas

- *desempilha_mult*(P,*k*) retira *k* elementos da pilha P

```
desempilha_mult(P,k)
   enquanto ((pilha não vazia) e (k ≠ 0)) faça
      desempilha(P)
      k ← k – 1
```

Qual a complexidade de pior caso de *desempilha_mult*(P,*k*) em uma pilha com *s* objetos?

Análise Agregada

Exemplo: Operações em Pilhas

```
desempilha_mult(P,k)
    enquanto ((pilha não vazia) e (k ≠ 0)) faça
        desempilha(P)
        k ← k – 1
```

O número de operações do *enquanto* é min(*s*,*k*).

Em cada iteração do *laço "enquanto"* é feita uma chamada a *desempilha*( ) com custo 1.

Portanto, o tempo de *desempilha_mult* é min(*s*,*k*).

Em uma pilha com *n* elementos o algoritmo é *O*(*n*).

Análise Agregada

Exemplo: Operações em Pilhas

```
desempilha_mult(P,k)
    enquanto ((pilha não vazia) e (k ≠ 0)) faça
        desempilha(P)
        k ← k – 1
```

O número de operações do *enquanto* é min(*s*,*k*).

Em cada iteração do *laço "enquanto"* é feita uma chamada a *desempilha*( ) com custo 1.

Portanto, o tempo de *desempilha_mult* é min(*s*,*k*).

Em uma pilha com *n* elementos o algoritmo é *O*(*n*).

Análise Agregada

Exemplo: Operações em Pilhas

```
desempilha_mult(P,k)
    enquanto ((pilha não vazia) e (k ≠ 0)) faça
        desempilha(P)
        k ← k – 1
```

O número de operações do *enquanto* é min(*s*,*k*).

Em cada iteração do *laço "enquanto"* é feita uma chamada a *desempilha*( ) com custo 1.

Portanto, o tempo de *desempilha_mult* é min(*s*,*k*).

Em uma pilha com *n* elementos o algoritmo é *O*(*n*).

Análise Agregada

Exemplo: Operações em Pilhas

```
desempilha_mult(P,k)
    enquanto ((pilha não vazia) e (k ≠ 0)) faça
        desempilha(P)
        k ← k – 1
```

O número de operações do *enquanto* é min(*s*,*k*).

Em cada iteração do *laço "enquanto"* é feita uma chamada a *desempilha*( ) com custo 1.

Portanto, o tempo de *desempilha_mult* é min(*s*,*k*).

Em uma pilha com *n* elementos o algoritmo é *O*(*n*).

Análise Agregada

Exemplo: Operações em Pilhas

Considere uma sequência de operações:
Empilha
Desempilha
Desempilha_mult

em uma pilha inicialmente vazia.

Análise Agregada

Exemplo: Operações em Pilhas

Considere uma sequência de operações:
***empilha*, *desempilha* e *desempilha_mult* em uma pilha inicialmente vazia.**

Pior caso de *desempilha_mult*( ) é $O(n)$.

Portanto, em uma sequência de n operações, teríamos $O(n^2)$, uma vez que são n operações com complexidade $O(n)$ cada.

Esse limite, entretanto, não é suficientemente estrito.

Análise Agregada

Exemplo: Operações em Pilhas

Considere uma sequência de operações:
empilha, *desempilha* e *desempilha_mult* em uma pilha inicialmente vazia.

Pior caso de *desempilha_mult*( ) é $O(n)$.

Portanto, em uma sequência de n operações, teríamos $O(n^2)$, uma vez que são n operações com complexidade $O(n)$ cada.

Esse limite, entretanto, não é suficientemente estrito.

Análise Agregada

Exemplo: Operações em Pilhas

Considere uma sequência de operações:
empilha, *desempilha* e *desempilha_mult* em uma pilha inicialmente vazia.

Pior caso de *desempilha_mult*( ) é $O(n)$.

Portanto, em uma sequência de n operações, teríamos $O(n^2)$, uma vez que são n operações com complexidade $O(n)$ cada.

Esse limite, entretanto, não é suficientemente estrito.

Análise Agregada

Exemplo: Operações em Pilhas

Cada objeto pode ser desempilhado, no máximo, uma vez para cada vez que for empilhado.

Em uma sequência de n operações *empilha*, *desempilha* e *desempilha_mult*, o número de vezes que um objeto pode ser desempilhado é, no máximo, n.

A sequência de n operações tem complexidade $O(n)$.

O custo amortizado de cada operação é, portanto,
$O(n)/n = O(1)$

Análise Agregada

Exemplo: Operações em Pilhas

Cada objeto pode ser desempilhado, no máximo, uma vez para cada vez que é empilhado.

Em uma sequência de *n* operações *empilha*, *desempilha* e *desempilha_mult*, o número de vezes que um objeto pode ser desempilhado é, no máximo, *n*.

A sequência de *n* operações tem complexidade $O(n)$.

O custo amortizado de cada operação é, portanto,
$O(n)/n = O(1)$

Análise Agregada

Exemplo: Operações em Pilhas

Cada objeto pode ser desempilhado, no máximo, uma vez para cada vez que é empilhado.

Em uma sequência de n operações *empilha*, *desempilha* e *desempilha_mult*, o número de vezes que um objeto pode ser desempilhado é, no máximo, n.

A sequência de n operações tem complexidade $O(n)$.

O custo amortizado de cada operação é, portanto,

$O(n)/n = O(1)$

Método do Contador

Trata a análise da sequência de operações como um sistema de débitos e créditos



Método do Contador

Trata a análise da sequência de operações como um sistema de **débitos** e **créditos**

Cada operação possui um certo número de **créditos** definido como ***Tempo Amortizado*** da operação.

Tempo amortizado = **número de créditos**

Método do Contador

Quando o custo amortizado da operação excede o número de créditos realmente gastos na operação, o excedente se transforma em créditos para futuras operações.

Deve-se mostrar que os créditos e débitos se compensam.



Método do Contador

Exemplo: Operações em Pilhas

Custos reais:

Empilha $\longrightarrow$ 1

Desempilha $\longrightarrow$ 1

Desempilha_mult $\longrightarrow$ min(s,k)

Considere os seguintes custos amortizados por operação

Empilha $\longrightarrow$ 2

Desempilha $\longrightarrow$ 0

Desempilha_mult $\longrightarrow$ 0

Método do Contador

Exemplo: Operações em Pilhas

Custos reais:

Empilha $\longrightarrow$ 1

Desempilha $\longrightarrow$ 1

Desempilha_mult $\longrightarrow$ min(s,k)

Considere os seguintes custos amortizados por operação

Empilha $\longrightarrow$ 2

Desempilha $\longrightarrow$ 0

Desempilha_mult $\longrightarrow$ 0

Método do Contador

Exemplo: Operações em Pilhas

Custos reais	Custos amortizados (créditos recebidos)
Empilha $\longrightarrow$ 1	Empilha $\longrightarrow$ 2
Desempilha $\longrightarrow$ 1	Desempilha $\longrightarrow$ 0
Desempilha_mult $\longrightarrow$ min(s,k)	Desempilha_mult $\longrightarrow$ 0

Método do Contador

Exemplo: Operações em Pilhas

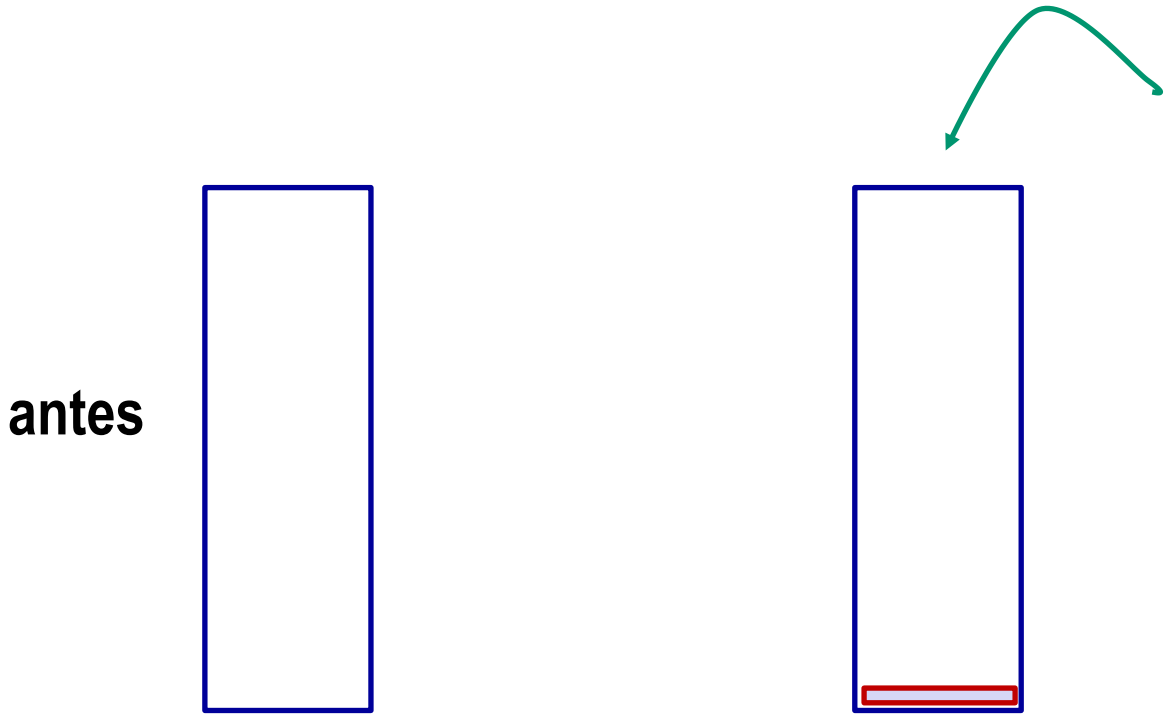
Devemos mostrar que é possível "*pagar*" por qualquer sequência de *n* operações com os créditos acumulados.

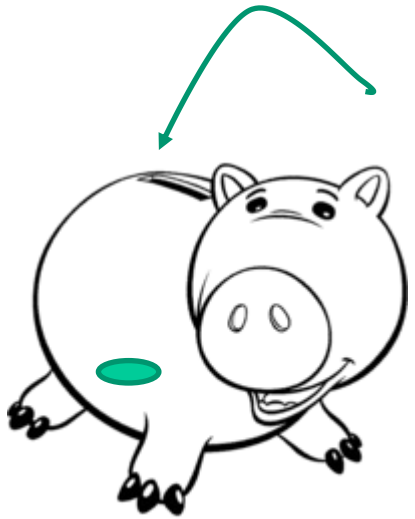


Método do Contador

Exemplo: Operações em Pilhas

Começando com a pilha vazia, ao se colocar 1 elemento na pilha, paga-se 1 unidade e economiza-se 1 unidade.



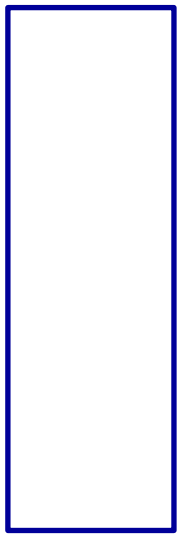


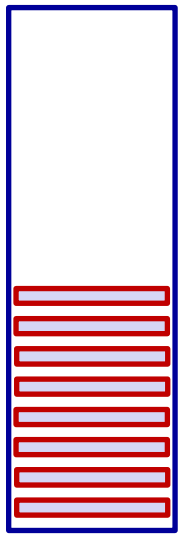
Método do Contador

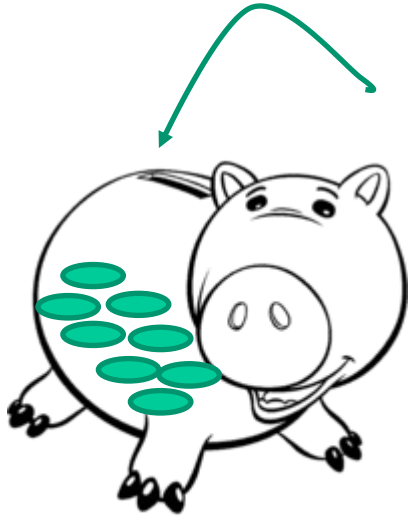
Exemplo: Operações em Pilhas

Quando a pilha tiver *i* elementos, terão sido economizados *i* créditos.

antes







Método do Contador

Exemplo: Operações em Pilhas

Quando a pilha tiver *i* elementos, terão sido economizados *i* créditos.

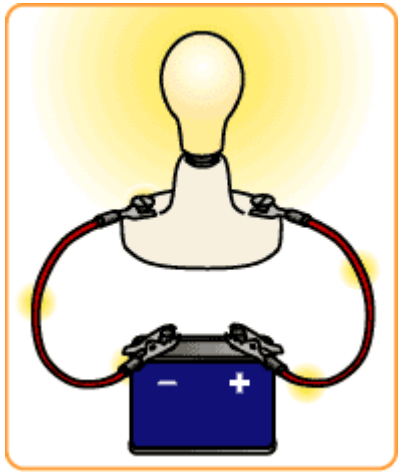
Portanto, existirão créditos suficientes para pagar por *i* remoções.

O custo amortizado da sequência é, portanto, $O(n)$.

Método do Potencial

Representa o trabalho pré-pago como "*energia potencial*" que pode ser liberada em futuras operações.

Função potencial $\Phi \rightarrow$ mapeia uma configuração da estrutura em um número real $\Phi(E)$, dito potencial de E.



Método do Potencial

O tempo amortizado, *a*, é definido como:

$$a = t + \Phi(E') - \Phi(E)$$

Onde

t é o tempo real da operação

E, E', são as configurações da estrutura antes e depois da operação.

Método do Potencial

Para *n* operações

$$\sum_{i=1}^{n} t_i = \sum_{i=1}^{n} \left(a_i - \Phi_i + \Phi_{i-1} \right) = \Phi_0 - \Phi_n + \sum_{i=1}^{n} a_i$$

onde

a_i e t_i são o tempo amortizado e o tempo real da i-ésima operação, respectivamente.

Φ_0, Φ_n são os potenciais antes da primeira e após a última operação.

Método do Potencial

Para *n* operações

$$\sum_{i=1}^{n}(a_i - \Phi_i + \Phi_{i-1}) =$$

$$a_1 - \Phi_1 + \Phi_0 + a_2 - \Phi_2 + \Phi_1 + a_3 - \Phi_3 + \Phi_2 + \cdots + a_n - \Phi_n + \Phi_{n-1}$$

Método do Potencial

Para *n* operações

$$\sum_{i=1}^{n}(a_i - \Phi_i + \Phi_{i-1}) =$$

$$a_1 - \Phi_1 + \Phi_0 + a_2 - \Phi_2 + \Phi_1 + a_3 - \Phi_3 + \Phi_2 + \cdots + a_n - \Phi_n + \Phi_{n-1}$$

$$\sum_{i=1}^{n}(a_i - \Phi_i + \Phi_{i-1}) = \Phi_0 - \Phi_n + \sum_{i=1}^{n} a_i$$

Método do Potencial

Obs:

***Os tempos reais não dependem dos estados intermediários*.**

Se o potencial final for maior ou igual ao potencial inicial, então a complexidade amortizada total pode ser usada como um limite superior do tempo real.

$$\sum_{i=1}^{n} t_i = \Phi_0 - \Phi_n + \sum_{i=1}^{n} a_i$$

Método do Potencial

Exemplo: Operações em Pilhas

Potencial da pilha: ***número de itens que ela contem***

Potencial inicial: $\Phi_0 = 0$

$\Phi(E)$ **é sempre não negativo**

Na *i*-ésima operação a pilha possui *p* elementos.

Realizar:

k* operações *desempilha

1 operação insere

} **tempo real,** $t_i = k + 1$

Método do Potencial

Exemplo: Operações em Pilhas

Potencial inicial da operação: $\Phi(E) = p$.

Potencial depois de realizada a operação: $\Phi(E) = p - k + 1$

O tempo amortizado de uma operação é : $a = t + \Phi(E') - \Phi(E)$

$$a = (k+1) + (p-k+1) - p = 2$$

Método do Potencial

Exemplo: Operações em Pilhas

Para *n* operações (iniciando com a pilha vazia)

$$\sum_{i=1}^{n} t_i = 0 - \Phi_m + \sum_{i=1}^{n} 2$$

potencial inicial

potencial final

tempo amortizado total

Tempo total real das operações será limitado superiormente por $2n$, portanto, $O(n)$.

Contador Binário

Exercício: Contador Binário

Qualquer número pode ter representação binária?

Exercício: Contador Binário

Qualquer número pode ter representação binária

***Base da Indução*: $1 = 2^0$**

Passo da Indução: Supor n representado por um conjunto de potências de 2 cuja soma é n. Se adicionarmos 2^0 a este conjunto teremos uma soma de $n + 1$.

Entretanto, podem existir duas cópias de 2^0. Para corrigir isto, enquanto tiver duas cópias de qualquer 2^i, deletar ambas e adicionar 2^{i+1}.

O valor da soma não se modifica, uma vez que $2^{i+1} = 2^i + 2^i$. Uma vez que o número de potências de 2 é decrementado de 1 unidade cada vez que isto é feito, o processo termina eventualmente. No final, teremos um conjunto de potências de 2 cuja soma é $n + 1$.

Exercício: Contador Binário

Qualquer número pode ter representação binária

***Base da Indução*: 1 = 2^0**

***Passo da Indução*: Supor *n* representado por um conjunto de potências de 2 cuja soma é *n*. Se adicionarmos 2^0 a este conjunto teremos *n* + 1.**

Entretanto, podem existir duas cópias de 2^0. Para corrigir isto, enquanto tiver duas cópias de qualquer 2^i, deletar ambas e adicionar 2^{i+1}.

O valor da soma não se modifica, uma vez que $2^{i+1} = 2^i + 2^i$. Uma vez que o número de potências de 2 é decrementado de 1 unidade cada vez que isto é feito, o processo termina eventualmente. No final, teremos um conjunto de potências de 2 cuja soma é *n* + 1.

Exercício: Contador Binário

Qualquer número pode ter representação binária

Base da Indução: $1 = 2^0$

Passo da Indução: Supor n representado por um conjunto de potências de 2 cuja soma é n. Se adicionarmos 2^0 a este conjunto teremos $n + 1$.

Entretanto, podem existir duas cópias de 2^0. Para corrigir isto, enquanto tiver duas cópias de qualquer 2^i, deletar ambas e adicionar 2^{i+1}.

O valor da soma não se modifica, uma vez que $2^{i+1} = 2^i + 2^i$. Uma vez que o número de potências de 2 é decrementado de 1 unidade cada vez que isto é feito, o processo termina eventualmente. No final, teremos um conjunto de potências de 2 cuja soma é $n + 1$.

Exercício: Contador Binário

Qualquer número pode ter representação binária

Base da Indução: $1 = 2^0$

Passo da Indução: Supor n representado por um conjunto de potências de 2 cuja soma é n. Se adicionarmos 2^0 a este conjunto teremos uma soma de $n + 1$.

Entretanto, podem existir duas cópias de 2^0. Para corrigir isto, enquanto tiver duas cópias de qualquer 2^i, deletar ambas e adicionar 2^{i+1}.

O valor da soma não se modifica, uma vez que $2^{i+1} = 2^i + 2^i$. Uma vez que o número de potências de 2 é decrementado de 1 unidade cada vez que isto é feito, o processo termina eventualmente. No final, teremos um conjunto de potências de 2 cuja soma é $n + 1$.

Exercício: Contador Binário

Qualquer número pode ter representação binária

Base da Indução: $1 = 2^0$

Passo da Indução: Supor n representado por um conjunto de potências de 2 cuja soma é n. Se adicionarmos 2^0 a este conjunto teremos uma soma de $n + 1$.

Entretanto, podem existir duas cópias de 2^0. Para corrigir isto, enquanto tiver duas cópias de qualquer 2^i, deletar ambas e adicionar 2^{i+1}.

O valor da soma não se modifica, uma vez que $2^{i+1} = 2^i + 2^i$.

Uma vez que o número de potências de 2 é decrementado de 1 unidade cada vez que isto é feito, o processo termina eventualmente. No final, teremos um conjunto de potências de 2 cuja soma é $n + 1$.

Exemplo: Contador Binário

- **Considere o algoritmo *Incremento* que recebe a entrada *B*, sendo *B* um vetor de bits, onde *B*[*i*] = 1 se e somente se 2^i aparece na soma.**

```
Incremento(B)
  i ← 0
  enquanto (B[i] = 1)
    B[i] ← 0
    i ← i + 1
  fim_enquanto
  B[i] ← 1
```

Exemplo: Contador Binário

- **Considere o algoritmo *Incremento* que recebe a entrada *B*, sendo *B* um vetor de bits, onde *B*[*i*] = 1 se e somente se 2^i aparece na soma.**

```
Incremento(B)
  i ← 0
  enquanto (B[i] = 1)
    B[i] ← 0
    i ← i + 1
  fim_enquanto
  B[i] ← 1
```

n = 11

Entrada:

1	0	1	1
3	2	1	0

← índices

Exemplo: Contador Binário

- **Considere o algoritmo *Incremento* que recebe a entrada *B*, sendo *B* um vetor de bits, onde *B*[*i*] = 1 se e somente se 2^i aparece na soma.**

```
Incremento(B)
  i ← 0
  enquanto (B[i] = 1)
    B[i] ← 0
    i ← i + 1
  fim_enquanto
  B[i] ← 1
```

n = 11

Entrada:				
1	0	1	1	
3	2	1	0	← índices

Iteração 1				
1	0	1	0	
3	2	1	0	

Iteração 2				
1	0	0	0	
3	2	1	0	

n = 12

Saída:				
1	1	0	0	
3	2	1	0	

Exercício: Contador Binário

Considere o algoritmo *Incremento* que recebe a entrada *B*, sendo *B* um vetor de bits, onde *B*[*i*] = 1 se e somente se 2^i aparece na soma.

```
Incremento(B)
  i ← 0
  enquanto (B[i] = 1)
    B[i] ← 0
    i ← i + 1
  fim_enquanto
  B[i] ← 1
```

Qual é a complexidade de pior caso do algoritmo *Incremento*?

Exercício: Contador Binário

Considere o algoritmo *Incremento* que recebe a entrada *B*, sendo *B* um vetor de bits, onde *B*[*i*] = 1 se e somente se 2^i aparece na soma.

```
Incremento(B)
  i ← 0
  enquanto (B[i] = 1)
    B[i] ← 0
    i ← i + 1
  fim_enquanto
  B[i] ← 1
```

Qual é a complexidade de pior caso do algoritmo *Incremento*?

Resposta: $O(\log n)$

Exemplo: Contador Binário

Suponha que o algoritmo *Incremento* é usado para contar de 0 a *n*. A complexidade é $O(n \log n)$.

Qual é a complexidade amortizada de cada execução do algoritmo *Incremento* para uma sequência de *n* chamadas deste algoritmo?

- **Utilizando:**

- **Análise Agregada**
- **Método do Contador**
- **Método do Potencial**

Exercício: Contador Binário

Observação:

Cada vez que *Incremento* é chamado não é necessário trocar os $\Theta(\log n)$ bits.

O bit menos significante *B*[0] troca toda vez.

***B*[1] só troca a cada duas vezes.**

***B*[2] a cada 4 vezes e, ...**

Em geral, o bit *B*[*i*] troca a cada 2^i vezes.

Exemplo: Contador Binário

- **Por exemplo: *n* = 9**

0	**0000**	**Troca de 0 bits**
1	**0001**	**Troca de 1 bit**
2	**0010**	**Troca de 2 bits**
3	**0011**	**Troca de 1 bit**
4	**0100**	**Troca de 3 bits**
5	**0101**	**Troca de 1 bit**
6	**0110**	**Troca de 2 bits**
7	**0111**	**Troca de 1 bit**
8	**1000**	**Troca de 4 bits**
9	**1001**	**Troca de 1 bit**

Total: Troca de 16 bits

Exercício: Contador Binário

Análise Agregada

Se começarmos de um vetor de 0s, uma sequência de *n* incrementos troca cada bit $\lfloor n/2^i \rfloor$ vezes.

Exercício: Contador Binário

Análise Agregada

Se começarmos de um vetor de 0s, uma sequência de *n* incrementos troca cada bit $\lfloor n/2^i \rfloor$ vezes.

O número total de trocas para a sequência é

$$\sum_{i=0}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < \sum_{i=0}^{\infty} \frac{n}{2^i} = 2n$$

***Em média*, cada execução de *Incremento* troca 2 bits.**

Exercício: Contador Binário

Análise Agregada

Se começarmos de um vetor de 0s, uma sequência de *n* incrementos troca cada bit $\lfloor n/2^i \rfloor$ vezes.

O número total de trocas para a sequência é

$$\sum_{i=0}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < \sum_{i=0}^{\infty} \frac{n}{2^i} = 2n$$

***Em média*, cada execução de *Incremento* troca 2 bits.**

O número de trocas total é $O(n)$.

O custo amortizado é $O(n)/n = O(1)$.

***O custo amortizado* de cada execução de *Incremento* é $O(1)$.**

Exercício: Contador Binário

Método do Contador

Custo amortizado = 2 (créditos) cada vez que que *Incremento* troca um único bit de 0 para 1.

Custo da troca de bit = 1

Exercício: Contador Binário

Método do Contador

Custo amortizado = 2 (créditos) cada vez que que *Incremento* troca um único bit de 0 para 1.

Custo da troca de bit = 1

Começando com o vetor *B* com zeros, ao se trocar um bit de 0 para 1, paga-se 1 unidade e economiza-se 1 unidade.

Assim, quando o bit correspondente for trocado para 0, existe crédito suficiente para a operação.

Exercício: Contador Binário

Por exemplo: *n* = 9

		CREDITOS NA OPERAÇÃO	PAGAMENTO	SALDO
0	0000	0	0	0
1	0001	2	1	1
2	0010	2	2	1
3	0011	2	1	2
4	0100	2	3	1
5	0101	2	1	2
6	0110	2	2	2
7	0111	2	1	3
8	1000	2	4	1
9	1001	2	1	2

Exercício: Contador Binário

Método do Potencial

Definir uma função potencial tal que $\Phi_0 = 0$ e $\Phi_i > 0$

$$\sum_{i=1}^{n} t_i = \sum_{i=1}^{n} a_i - \Phi_n \leq \sum_{i=1}^{n} a_i$$

Exercício: Contador Binário

Método do Potencial

$$\sum_{i=1}^{n} t_i = \sum_{i=1}^{n} a_i - \Phi_n \leq \sum_{i=1}^{n} a_i$$

Função potencial após a *i*-ésima operação igual ao número de bits 1.

Inicialmente, todos os bits são 0 e, portanto, $\Phi_0 = 0$.
Além disso, $\Phi_i > 0$, para todo $i > 0$.

t_i = número de bits trocados de 0 para 1 + número de bits trocados de 1 para 0 na *i*-ésima operação.

$\Phi_i - \Phi_{i-1}$ = número de bits trocados de 0 para 1 - número de bits trocados de 1 para 0 na *i*-ésima operação.

Exercício: Contador Binário

Método do Potencial

t_i = número de bits trocados de 0 para 1 + número de bits trocados de 1 para 0 na *i*-ésima operação.

$\Phi_i - \Phi_{i-1}$ = número de bits trocados de 0 para 1 - número de bits trocados de 1 para 0 na *i*-ésima operação.

O custo amortizado do *i*-ésimo incremento é:

$a_i = t_i + \Phi_i - \Phi_{i-1}$

$a_i = 2 \times$ número de bits trocados de 0 para 1

Uma vez que *Incremento* troca apenas um bit de 0 para 1, o custo amortizado por operação é 2, portanto, $O(1)$.

Estruturas Autoajustáveis

Estruturas que reagem modificando-se um função de alguma operação realizada.



Listas Autoajustáveis

- **Em um lista não ordenada, um novo elemento é inserido na última posição da lista.**

- **Suponha que esse elemento seja acessado muito mais frequentemente que aqueles que se localizam em posições anteriores na lista.**

- **Então, pode ser interessante, colocar esse elemento mais próximo ao início da lista.**

Listas Autoajustáveis

- **Uma lista linear (não ordenada) autoajustável deve permutar a ordem dos seus nós, a fim de colocar em posições mais próximas do início, os nós mais procurados.**

- **Aplicações:**

- **Situações onde a simplicidade do algoritmo não compense o uso de estruturas mais elaboradas.**
- **Autoajuste em listas encadeadas que resolvem o problema de colisões por encadeamento exterior em tabelas de dispersão.**

Métodos de Autoajuste

- **Mover para Frente (MF)**
- **Transposição (TR)**
- **Mover para frente k posições (MF-k)**
- **Contador de Frequência (CF)**
- **Métodos Híbridos**

Mover para Frente (MF)

- **O nó encontrado é movido para o início da lista (caso não seja o primeiro)**

Mover para Frente (MF)

- **O nó encontrado é movido para o início da lista (caso não seja o primeiro)**



Lista inicial

Buscar: 17

Mover para Frente (MF)

- **O nó encontrado é movido para o início da lista (caso não seja o primeiro)**



Lista inicial

Buscar: 17



Movimentação

Mover para Frente (MF)

- **O nó encontrado é movido para o início da lista (caso não seja o primeiro)**

👍 **sem problemas para alocação encadeada**

👎 **na alocação sequencial é necessário movimentar todos os nós antes dele.**

👎 **quando um nó com baixa probabilidade de acesso é movimentado para o início da lista, ele incrementa os custos das futuras operações.**

Transposição (TR)

- **O nó encontrado é trocado com o que o precede.**

Transposição (TR)

- **O nó encontrado é trocado com o que o precede.**



Lista inicial

Buscar: 17



Movimentação

Transposição (TR)

- **O nó encontrado é trocado com o que o precede.**

Vantajoso para alocação sequencial.

Um nó se aproxima do início da lista se for acessado frequentemente.

Mover para frente k posições (MF-k)

- **O nó encontrado é trocado com os k nós que o precedem.**

Método intermediário entre MF e TR.

Após o acesso, o nó é movido k posições para frente (se possível).

Se *k'* é a distância do nó ao início da lista, então:

MF equivale a MF-*k'*

TR equivale a MF-1

Mover para frente k posições (MF-k)

- **O nó encontrado é trocado com os k nós que o precedem.**



Lista inicial

Buscar: 17



Movimentação

k = 3

Contador de Frequência (CF)

- **Cada nó possui um contador de frequência que é incrementado a cada acesso.**
- **O nó é movimentado para uma posição que preceda os demais nós com contadores menores.**

A lista fica em ordem não crescente do contador.

A tabela fica ordenada pela *probabilidade de acesso*.

Contador de Frequência (CF)

- **Cada nó possui um contador de frequência que é incrementado a cada acesso.**

12	9	7	1	1	1	1			
7	90	6	34	22	5	17			

Contador de frequência

Lista inicial

Buscar: 17

12	9	7	2	1	1	1			
7	90	6	17	34	22	5			

Movimentação

Métodos Híbridos

- **Utilizam dois ou mais métodos vistos em conjunto.**





Método Admissível

Após um acesso a um nó, o método move o nó acessado zero ou mais posições para frente.

Para um método admissível *H* e uma sequência, *s*, de operações:

$C_H(s)$: tempo total real da sequência de operações

$E_H(s)$: número total de trocas realizadas

Análise do Método MF

O tempo do método MF é limitado superiormente por 2 vezes o tempo de qualquer outro método admissível.

Teorema

Para qualquer método admissível *H*, e qualquer sequência *s* de dimensão *m* composta por operações de busca com sucesso e inserções, a partir da lista vazia, tem-se:

$$C_{MF}(s) \leq 2\ C_H(s) - E_H(s) - m$$

Análise do Método MF

- **Prova. Considere a sequência *s* aplicada a uma lista e a utilização dos dois métodos resultando nas listas MF e H.**

- Utilizando o Método do Potencial
- Função potencial : *número total de inversões na lista MF em relação a H*
- Uma inversão em uma lista em relação à outra é um par não ordenado (*i*,*j*) tal que *i* precede *j* em uma das listas e *i* sucede *j* na outra.

Análise do Método MF

- **Prova. Considere a sequência *s* aplicada a uma lista e a utilização dos dois métodos resultando nas listas MF e H.**

Utilizando o Método do Potencial

- **Função potencial : *número total de inversões na lista MF em relação a H***
- **Uma inversão em uma lista em relação à outra é um par não ordenado (*i*,*j*) tal que *i* precede *j* em uma das listas e *i* sucede *j* na outra.**

Análise do Método MF

Exemplo de inversões

- ***A*** **= (1, 5, 4, 2, 8, 6, 7, 3)**
- ***B*** **= (5, 2, 3, 7, 1, 8, 4, 6)**

13 inversões considerando *A* em relação a *B*:

(1,2), (1,3), (1,5), (1,7), (2,4), (3,4), (3,6), (3,7), (3,8), (4,7), (4,8), (6,7), (7,8)

Análise do Método MF

Inserção

- **A chave é inserida no final da lista.**
- **A operação não altera o número de inversões.**

Análise do Método MF

Busca com Sucesso

- **Considere *x* (a chave de busca) o *j*-ésimo elemento em MF e o *k*-ésimo elemento em *H*.**

- **Dado um elemento *y* existem duas situações em relação ao par (*x*,*y*) e a lista MF:**

- ***y* precede *x***
- ***y* não precede *x***

Análise do Método MF

Busca com Sucesso

- ***y* não precede *x***

  movimentar *x* não contribui para alteração do número de inversões.

Análise do Método MF

Considere *y* um elemento que precede *x* em MF.

Existem dois casos:

1. *y* também precede *x* em *H*.

2. *y* sucede *x* em *H*.

Análise do Método MF

Considere *p* o número de elementos *y* que precedem *x* em MF e o sucedem em H.

Análise do Método MF

Considere *p* o número de elementos *y* que precedem *x* em MF e o sucedem em H.

***x* é o *j*-ésimo elemento em MF e o *k*-ésimo elemento em *H*.**

- **Após o acesso *x* é movido para o início da lista MF e passa a frente de *j*-1 elementos.**
- **Então *p* inversões são destruídas e *j*-1-p novas inversões são criadas.**
- **Com *x* na frente em MF, todos os elementos que o precedem em H formam agora uma inversão.**

Análise do Método MF

- **Quando *x* é movido e_i posições em H, dependendo do método sendo utilizado, exatamente e_i inversões são destruídas.**

- **Essa movimentação não cria qualquer nova inversão, pois *x* ocupa a primeira posição em MF.**

Análise do Método MF

Lembrando: Tempo amortizado pelo método do Potencial é:

$$a_i = t_i + \Phi(E') - \Phi(E)$$

Análise do Método MF

- **O tempo real t_j para acesso a *x* em MF é *j*.**

***x* é o *j*-ésimo elemento em MF**

Análise do Método MF

- **O tempo real t_i para acesso a *x* em MF é *j*.**
- **A diferença de potencial é o resultado entre as inversões criadas e removidas.**

$$a_i = j + (j - 1 - p) - p - e_i$$

criadas removidas

Análise do Método MF

- **O tempo real t_i para acesso a x em MF é j.**
- **A diferença de potencial é o resultado entre as inversões criadas e removidas.**

$$a_i = j + (j - 1 - p) - p - e_i$$

- **Somando +1 e -1**

$$a_i = 2\,(j - 1 - p) - e_i + 1$$

Análise do Método MF

- O tempo real t_i para acesso a x em MF é j.
- A diferença de potencial é o resultado entre as inversões criadas e removidas.

$$a_i = j + (j - 1 - p) - p - e_i$$

- Somando +1 e -1

$$a_i = 2\,(j - 1 - p) - e_i + 1$$

- Sabemos que $(j - 1 - p) \leq k - 1$

k-1 nós precediam x em H

Análise do Método MF

- **O tempo real t_i para acesso a x em MF é j.**
- **A diferença de potencial é o resultado entre as inversões criadas e removidas.**

$$a_i = j + (j - 1 - p) - p - e_i$$

- **Somando +1 e -1**

$$a_i = 2\,(j - 1 - p) - e_i + 1$$

- **Sabemos que $(j - 1 - p) \leq k - 1$**

- **Então,** $a_i \leq 2\,(k - 1) - e_i + 1 = 2\,k - e_i - 1$

Análise do Método MF

- **Observe que *k* é o tempo real de acesso de *x* em H.**

- **Sendo $\Phi_0 = 0$ e $\Phi_m \geq 0$, o somatório do tempo real de todas as operações é:**

$$C_{MF}(s) = \sum_{i=1}^{m} t_i$$

Análise do Método MF

$$C_{MF}(s) = \sum_{i=1}^{m} t_i$$

$$\sum_{i=1}^{n} t_i = \Phi_0 - \Phi_n + \sum_{i=1}^{n} a_i$$

$$C_{MF}(s) = \Phi_0 - \Phi_n + \sum_{i=1}^{n} a_i$$

Análise do Método MF

$$C_{MF}(s) = \Phi_0 - \Phi_m + \sum_{i=1}^{m} a_i$$

Φ_0: **0**; Φ_m: **≥ 0**

$$\sum_{i=1}^{m} a_i \leq \sum_{i=1}^{m} (2k - e_i - 1)$$

$$C_{MF}(s) \leq \sum_{i=1}^{m} (2k - e_i - 1)$$

Análise do Método MF

$$C_{MF}(s) \leq \sum_{i=1}^{m} (2k - e_i - 1)$$

$$C_{MF}(s) \leq 2\, C_H(s) - E_H(s) - m$$

Árvore de Difusão

- **A árvore de difusão é uma *árvore binária de busca autoajustável*.**
- **Ideia: Efetuar operações de ajuste após cada acesso a um nó**
- **Asseguram *Complexidade Amortizada* em $O(\log n)$**



Operações de Difusão

- **Iniciam em um certo nó *q* e se propagam até a raiz *r* da árvore *T***

- **A difusão de *q* depende da posição do seu *pai* e do seu *avô* (caso existam)**

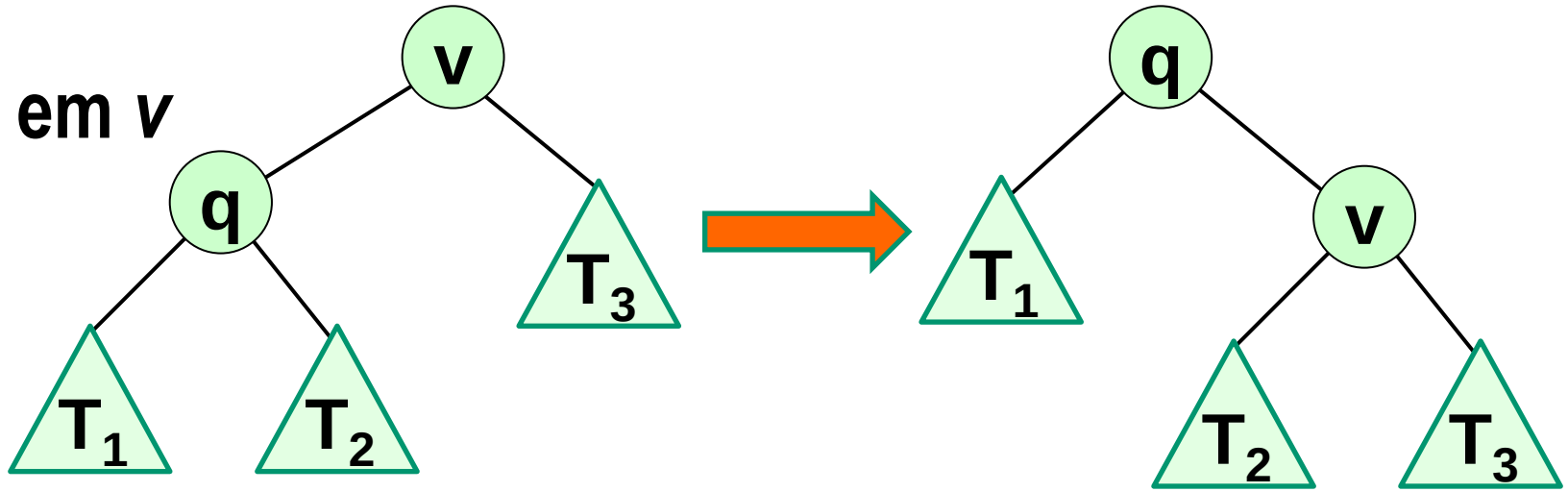
- **3 casos a considerar:**

- ***q* é raiz**
- ***q* é filho da raiz**
- ***q* é filho de *v* (*v* não é raiz)**

Operações de Difusão

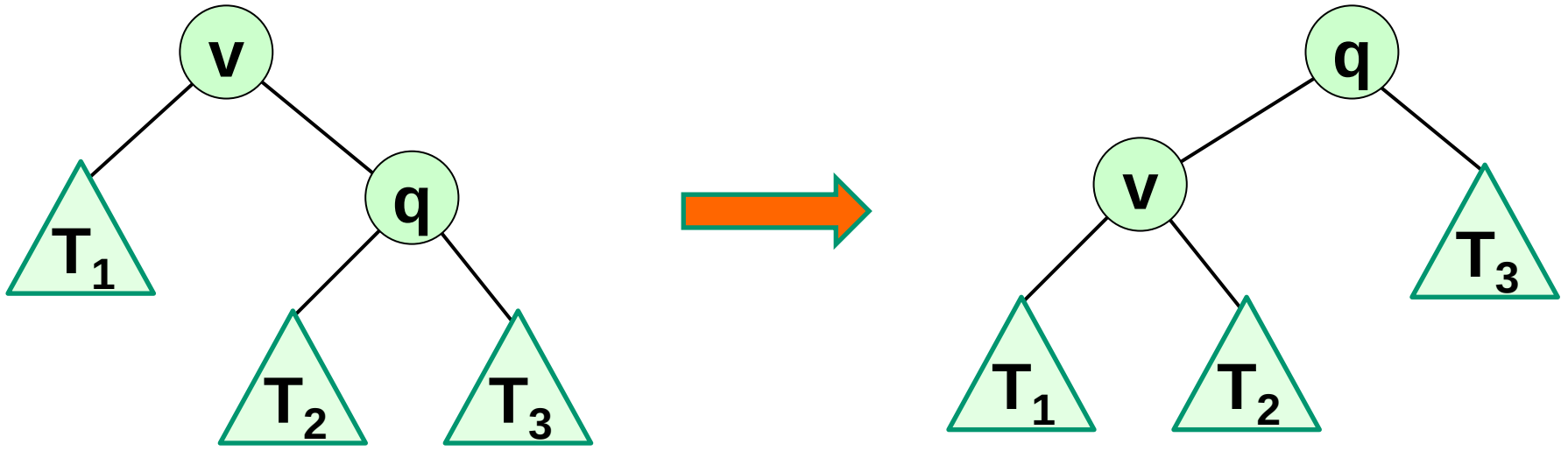
- **Caso 1**. ***q* = *r***
- **Nada a efetuar.**

- **Caso 2**. ***q* é filho de *v* = *r***
- **2.1**. ***q* é filho esquerdo**
- **Efetuar *Rotação Direita* em *v***



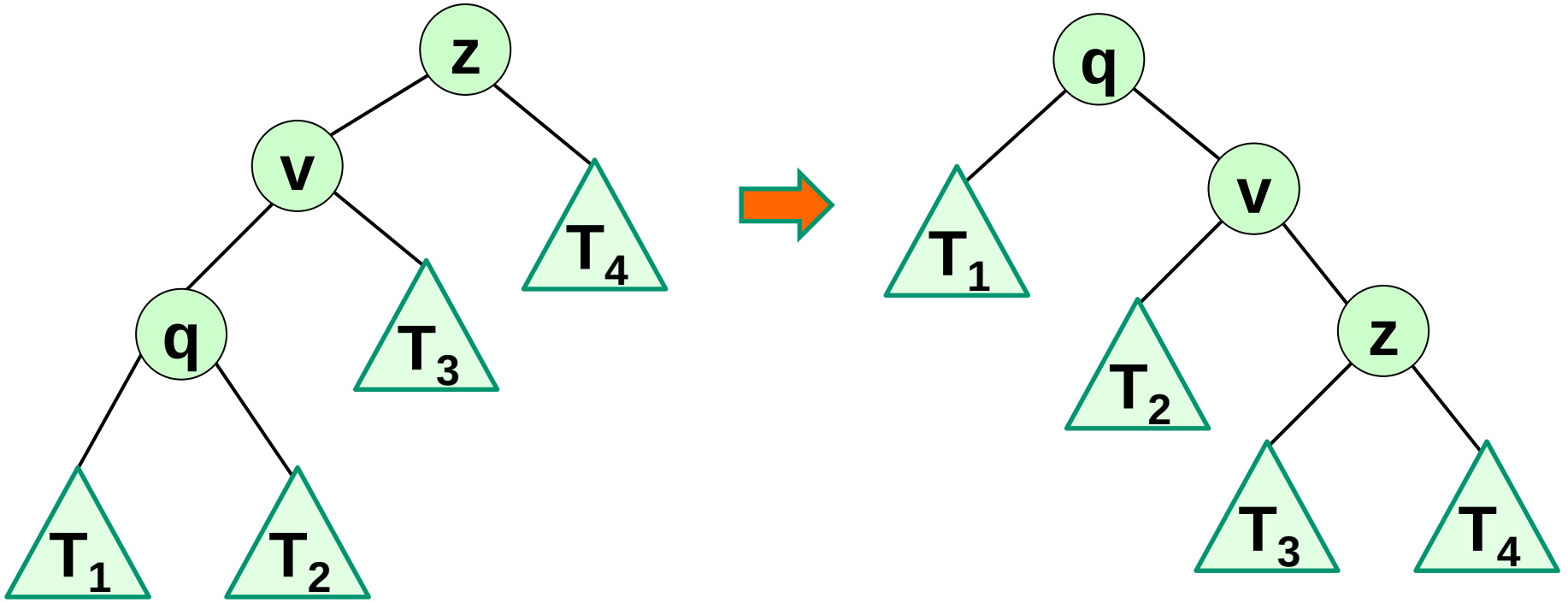
Operações de Difusão

- **Caso 2**. ***q*** **é filho de** ***v*** **=** ***r***
- **2.2**. ***q*** **é filho direito**
- **Efetuar** ***Rotação Esquerda*** **em** ***v***



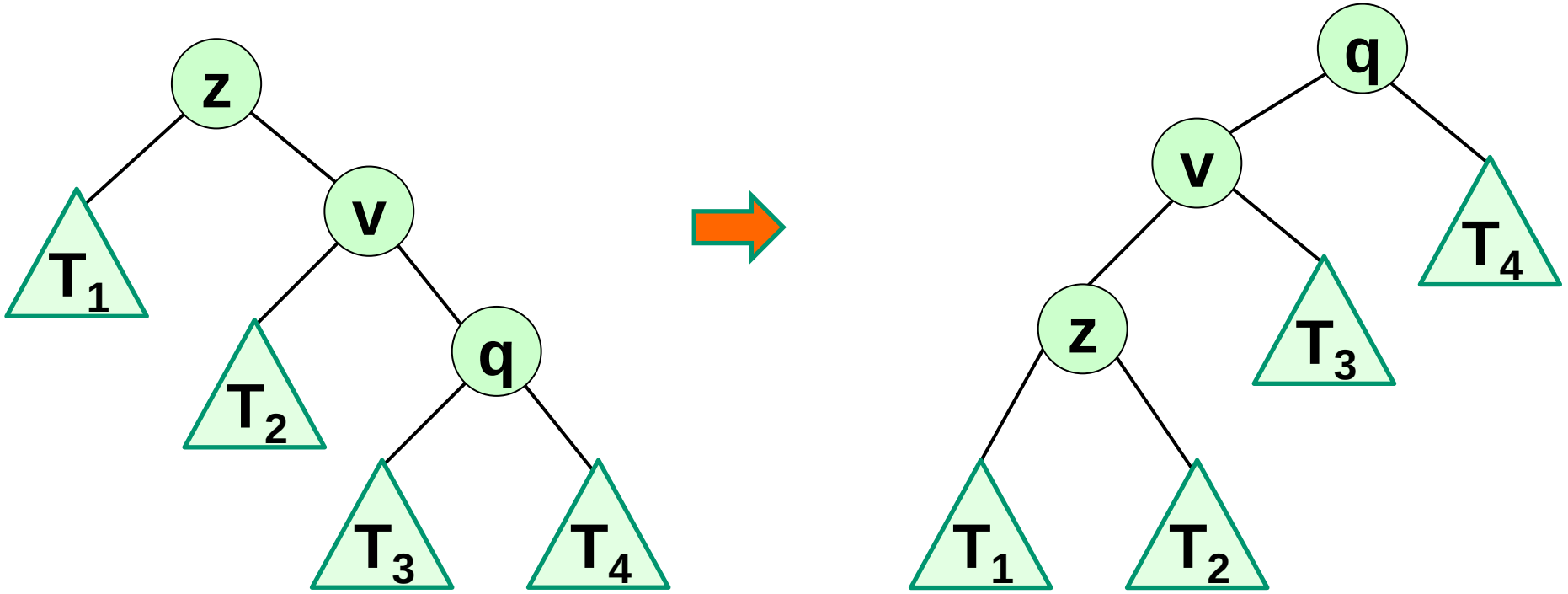
Operações de Difusão

- **Caso 3**. ***q*** **é filho de** ***v*** **(não raiz) - Considerar nó** ***z*** **avô de** ***q***
- **3.1**. ***q,v*** **são filhos esquerdos**
- **Rotação direita em** ***z*** **e, em seguida, em** ***v***



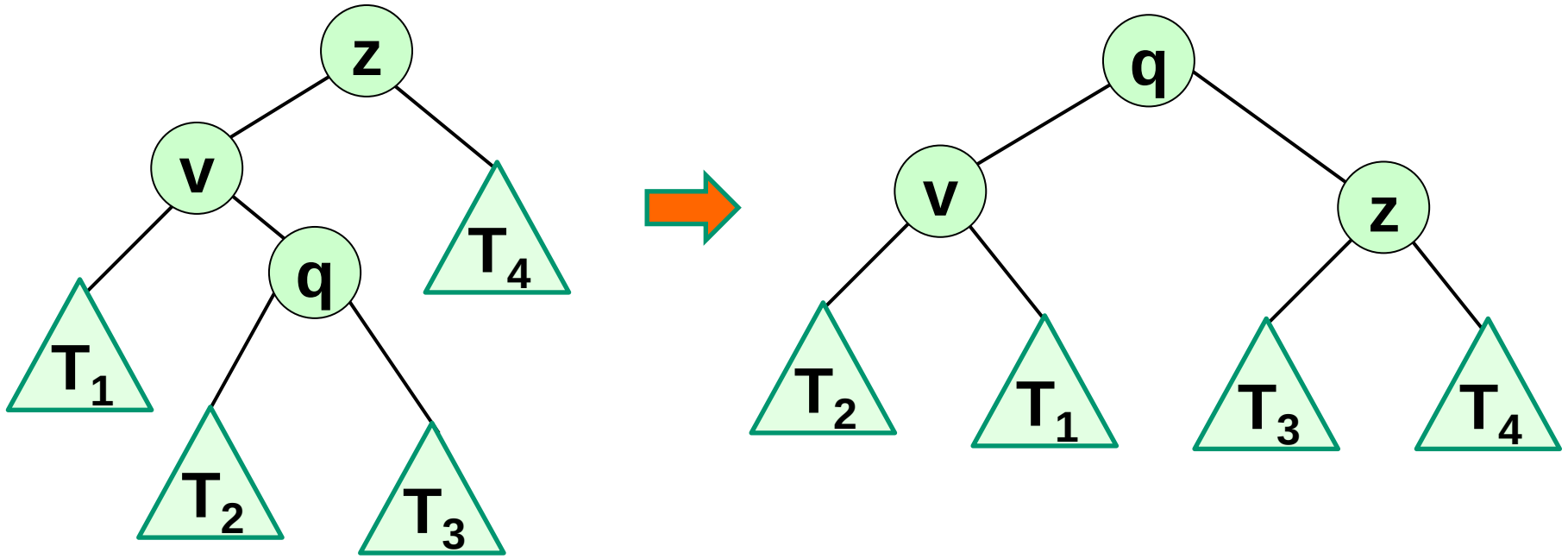
Operações de Difusão

- **3.2. *q*,*v* são filhos direitos**
- **Rotação esquerda em *z* e, em seguida, em *v***



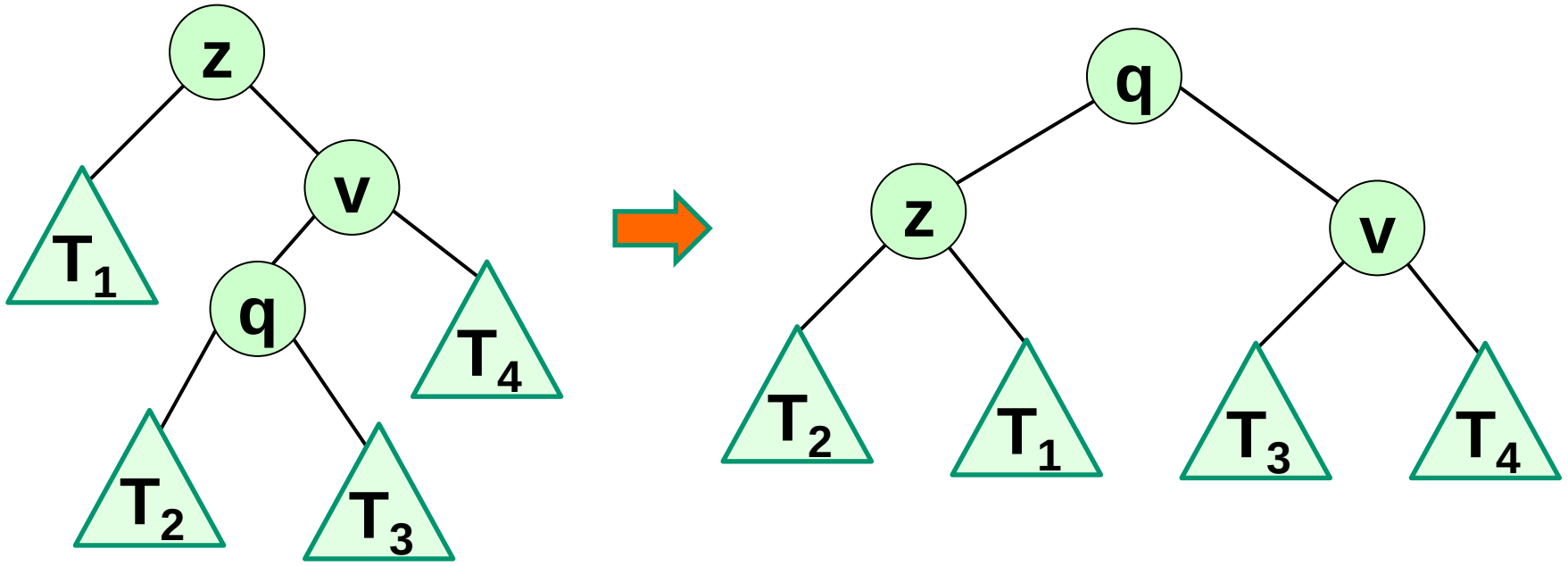
Operações de Difusão

- **3.3. *q* é filho direito e *v* é filho esquerdo**
- **Rotação dupla esquerda em *z***



Operações de Difusão

- **3.3. *q* é filho esquerdo e *v* é filho direito**
- **Rotação dupla direita em *z***



Árvores de Difusão

- **Nos casos 1 e 2, *q* se torna raiz da árvore.**

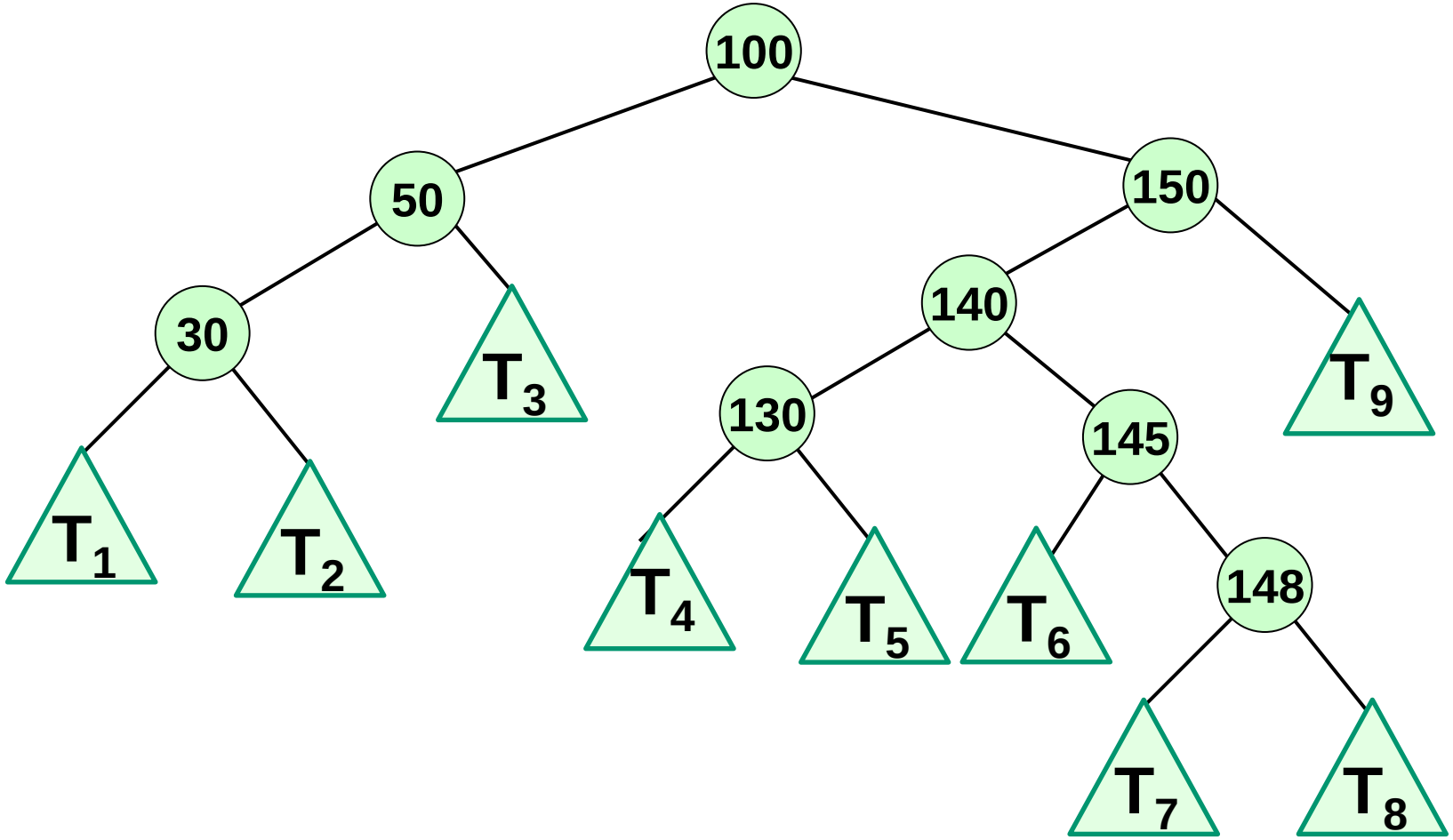
- **Nos casos 3, a distância de *q* para a raiz da árvore diminui de 2 unidades.**

- **A difusão completa de *q* repete o processo até que *q* seja raiz da árvore.**

```
enquanto (q ≠ r)
    difusão(q)
```

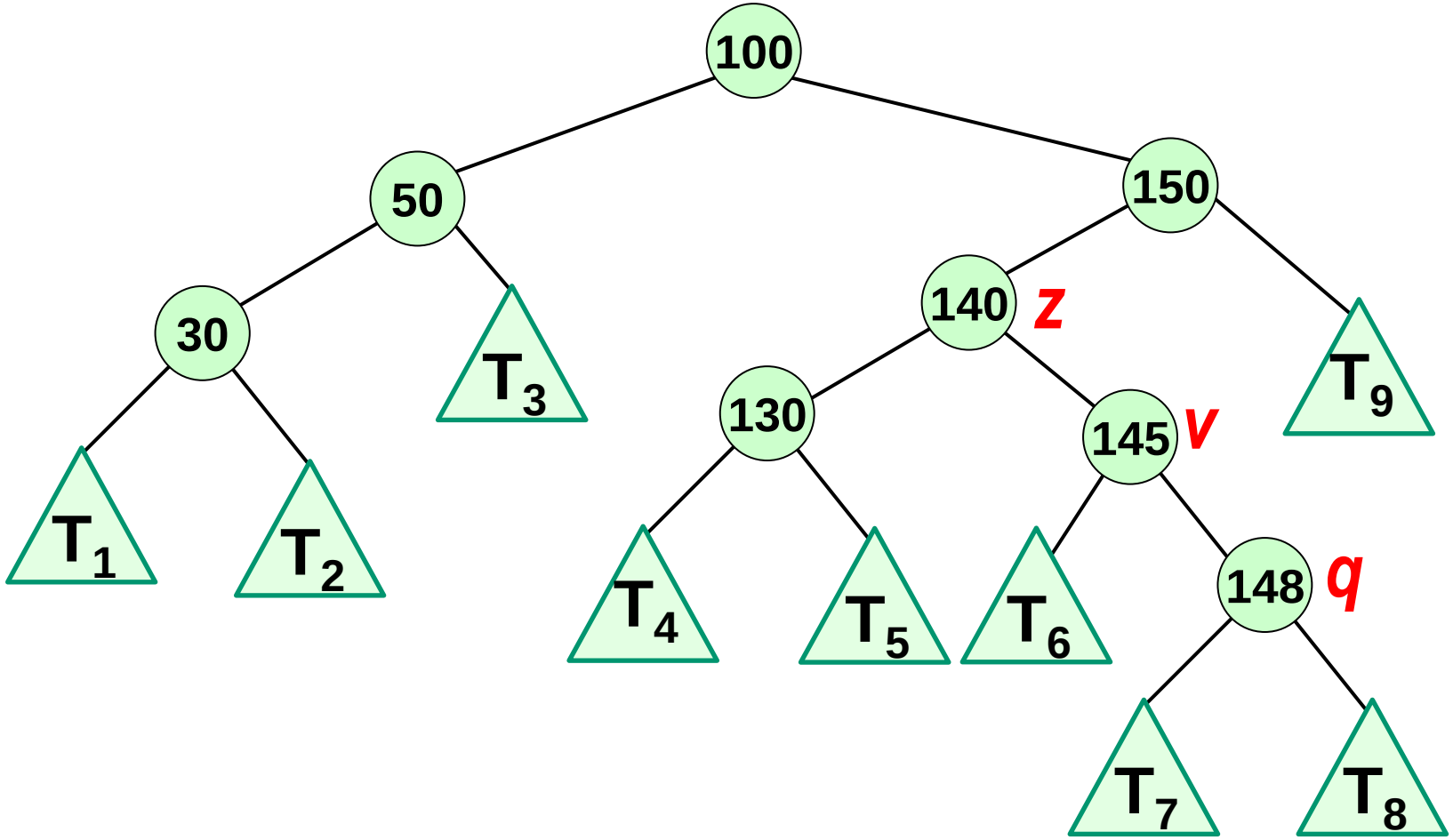
Árvores de Difusão

- **Exemplo: Difusão completa do nó 148**



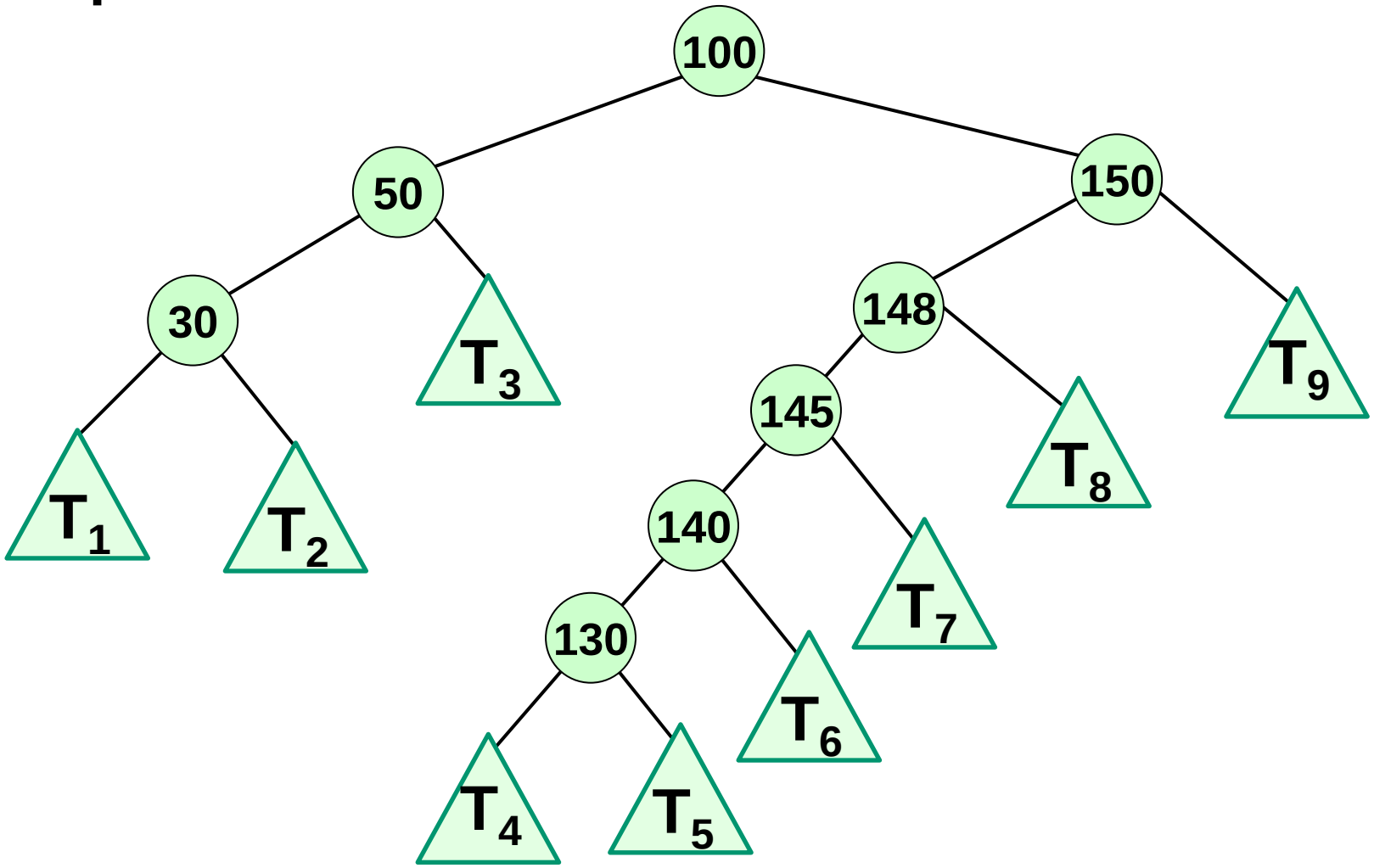
Árvores de Difusão

- **Exemplo: Difusão completa do nó 148**



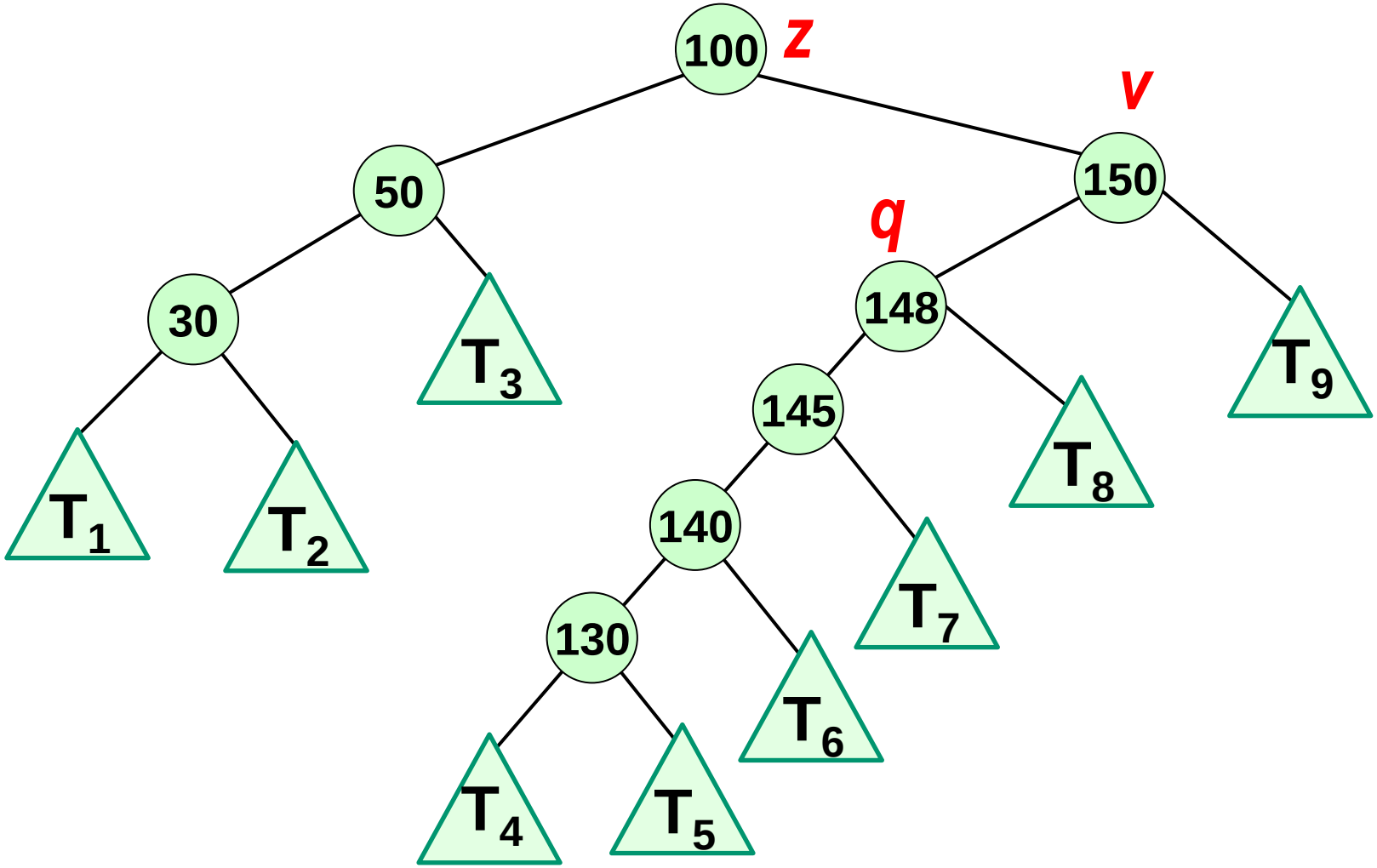
Árvores de Difusão

- **Exemplo: Difusão completa do nó 148**



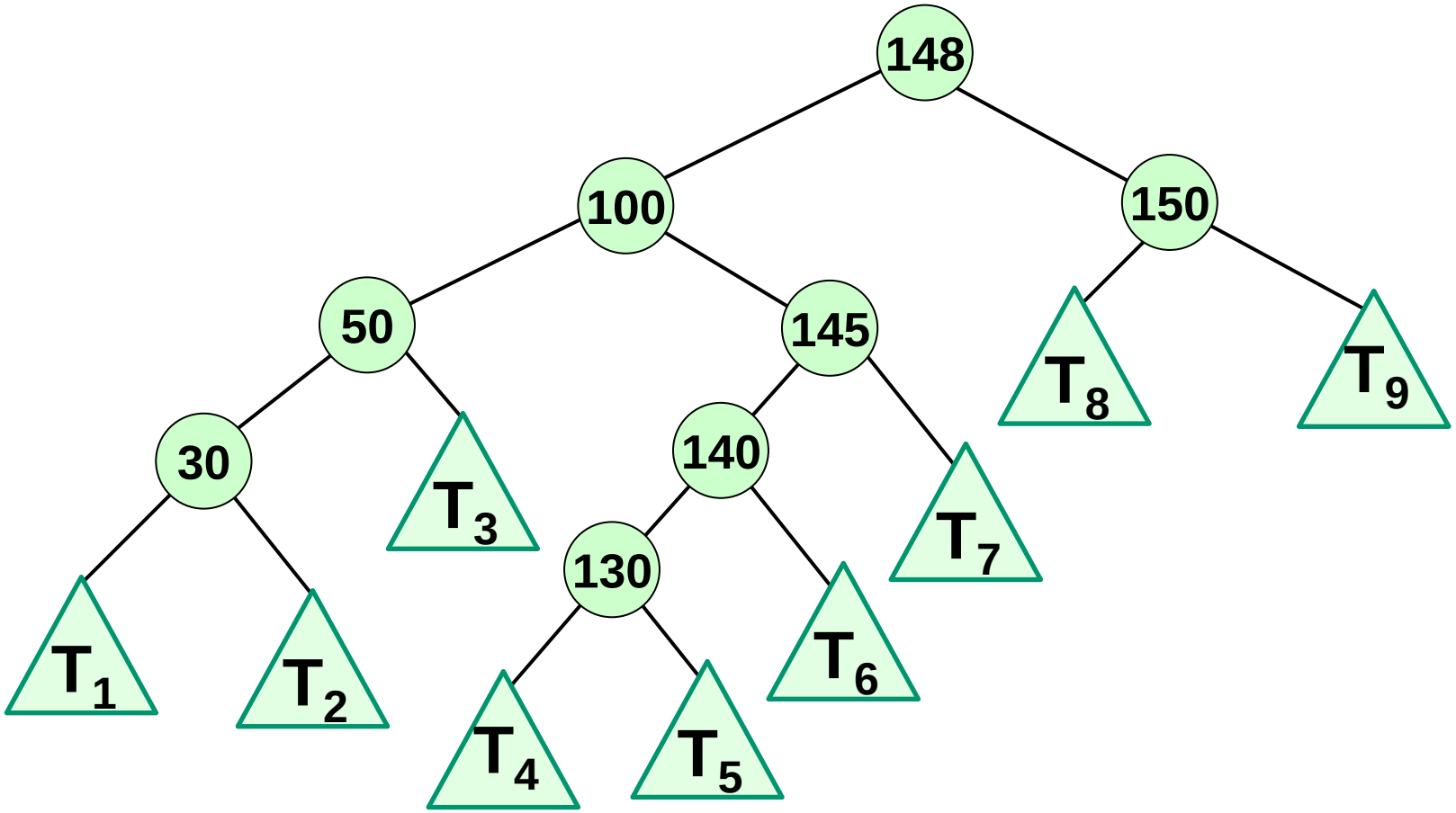
Árvores de Difusão

- **Exemplo: Difusão completa do nó 148**



Árvores de Difusão

- **Exemplo: Difusão completa do nó 148**



Busca em Árvores de Difusão

- ***Sucesso*:**
- **Após acessar o nó *q* solicitado, efetuar difusão completa de *q*.**

- ***Falha*:**
- **Após verificar que o nó não se encontra na árvore, efetuar difusão completa do último nó diferente de nulo acessado na busca.**

Inserção e Remoção em Árvores de Difusão

- **Inserção**

- **Após inserir o novo nó *q*, efetuar difusão completa de *q*.**

- **Remoção**

- **Após remover *q*, efetuar difusão completa do pai de *q* (se existir)**

Complexidade

- ***Objetivo*:**

  Determinar o tempo total necessário para efetuar uma sequência de operações no seu pior caso.

- ***Método*:**

  Método do Potencial

Complexidade

Função Potencial

- A cada nó *v* é associado um peso positivo $w(v)$.
- O valor do peso é arbitrário. Não se altera depois de definido.

- Define-se $W(v)$ como a soma dos pesos dos nós na subárvore de raiz *v*.
- Define-se o posto de *v*, $p(v) = \lfloor \log W(v) \rfloor$.

- O *Potencial da Árvore* é a soma dos postos de todos os seus nós.

Complexidade

- **Potencial da árvore**

$$\sum_{v} p(v) = \sum_{v} \lfloor \log W(v) \rfloor$$

Complexidade

Teorema

- **O tempo amortizado para uma difusão completa de um nó *q* em uma árvore de raiz *r* é, no máximo,**

$$1 + 3[p(r) - p(q)]$$

Prova

(Szwarcfiter e Markenzon, 1994)

Complexidade

- Considerando $w(v) = 1$, tem-se $p(v) = \lfloor \log W(v) \rfloor$, onde $W(v)$ representa o número de nós na subárvore de raiz v.
- Assim, $p(r) = \lfloor \log n \rfloor$ onde r é a raiz da árvore e n é o número de nós na árvore.

- Pelo lema anterior, sabe-se que o custo amortizado total da difusão do nó q é $1 + 3p(r) - 3p(q)$

Potencial de q depois da difusão, uma vez que q passa a ser a raiz

Complexidade

- Portanto, o custo total amortizado da difusão do nó *q* (em uma operação de busca, inserção ou remoção) é $O(\log n)$.

Complexidade

- **Teorema**

- O tempo total necessário para efetuar uma sequência de ***m*** **operações** de busca, inserção ou exclusão em uma árvore de difusão, inicialmente vazia, é ***O(mlogn)*** onde *n* é o número de inclusões realizadas.



Prova

(Szwarcfiter e Markenzon, 1994)

Operações em Conjuntos Disjuntos

Elementos do conjunto pertencem a um universo com *n* elementos.

- **- *gerar*(*x*) : Cria um conjunto unitário {*x*} para cada elemento do universo, sendo *x* também o nome do conjunto.**
- **- *buscar*(*x*): Determina o nome do conjunto ao qual o elemento *x* pertence**
- **- *fundir*(*a*,*b*): Determina a união dos conjuntos disjuntos cujos nomes são *a* e *b*. O resultado da fusão é atribuído ao primeiro desses conjuntos.**

Operações em Conjuntos Disjuntos

Elementos do conjunto pertencem a um universo com *n* elementos.

- **- *gerar*(*x*) : Cria um conjunto unitário {*x*} para cada elemento do universo, sendo *x* também o nome do conjunto.**
- **- *buscar*(*x*): Determina o nome do conjunto ao qual o elemento *x* pertence**
- **- *fundir*(*a*,*b*): Determina a união dos conjuntos disjuntos cujos nomes são *a* e *b*. O resultado da fusão é atribuído ao primeiro desses conjuntos.**

Operações em Conjuntos Disjuntos

Elementos do conjunto pertencem a um universo com *n* elementos.

- **- *gerar*(*x*) : Cria um conjunto unitário {*x*} para cada elemento do universo, sendo *x* também o nome do conjunto.**
- **- *buscar*(*x*): Determina o nome do conjunto ao qual o elemento *x* pertence**
- **- *fundir*(*a*,*b*): Determina a união dos conjuntos disjuntos cujos nomes são *a* e *b*. O resultado da fusão é atribuído ao primeiro desses conjuntos.**

Representação de Conjuntos Disjuntos

Representação em Listas Encadeadas

A chave no primeiro nó é o representante do conjunto.

Cada nó contem uma chave, um ponteiro para o primeiro nó e um ponteiro para o nó seguinte.

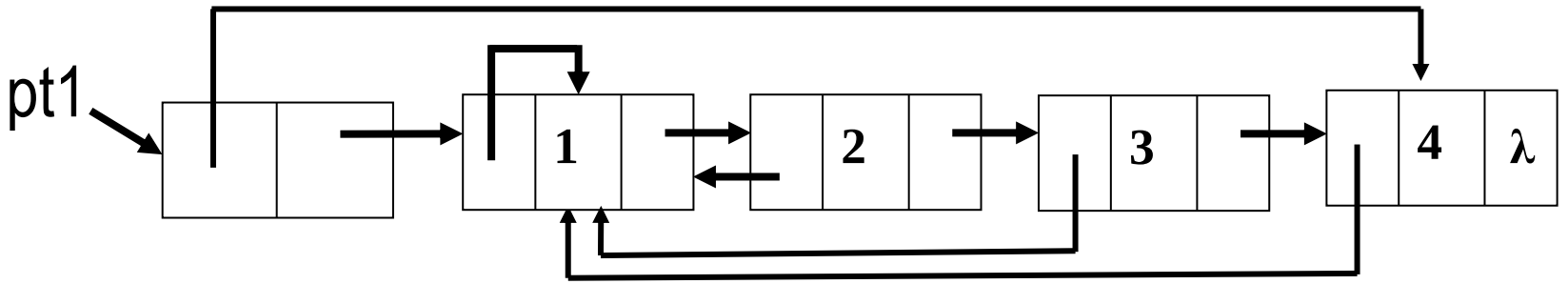
Cada lista tem um nó cabeça com dois campos: início e fim.

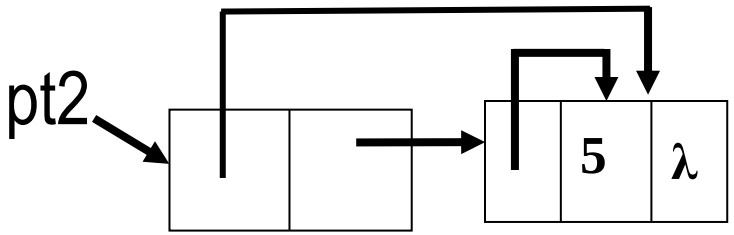
Os elementos do conjunto aparecem em uma ordem qualquer na lista (assumindo que o primeiro elemento é o representante).

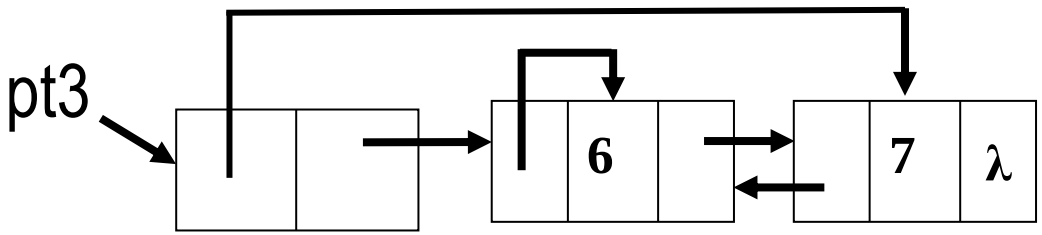
Representação de Conjuntos Disjuntos

Exemplo - Representação em Listas Encadeadas

Conjuntos: {1,2,3,4}, {5}, {6,7}

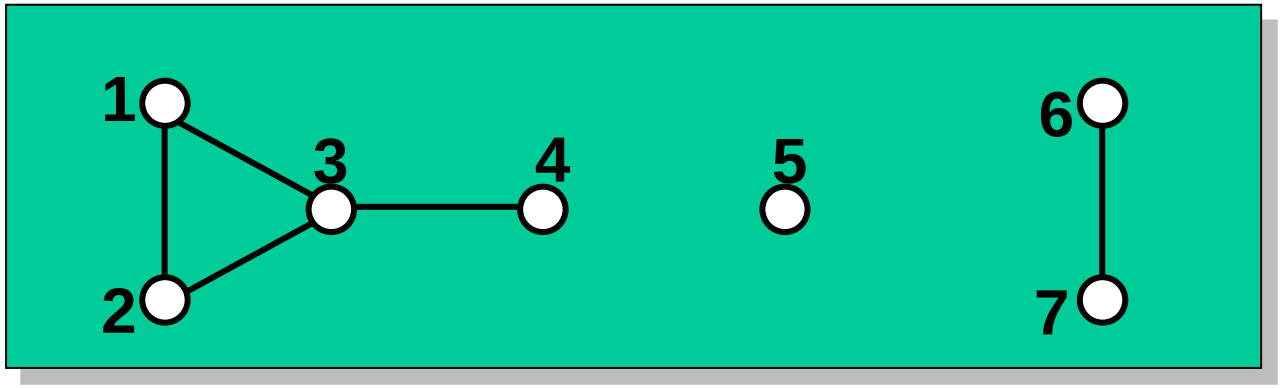






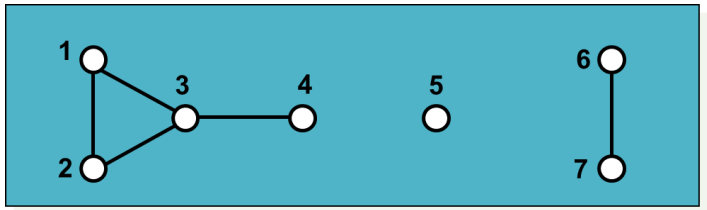
Uma Aplicação em Grafos

Determinar componentes conexos



Uma Aplicação em Grafos

Determinar componentes conexos



Procedimento Componente_conexo(G = (V,E))

para cada v $\in$ V

gerar(v)

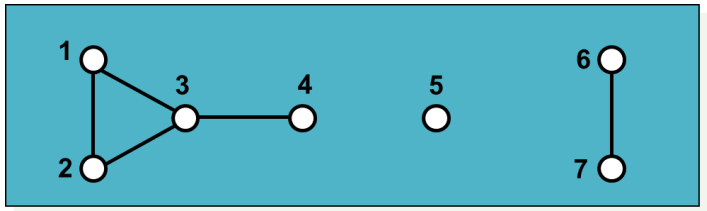
para cada (u,v) $\in$ E

se busca(u) $\neq$ busca(v)

fundir(u,v)

Uma Aplicação em Grafos

Determinar componentes conexos



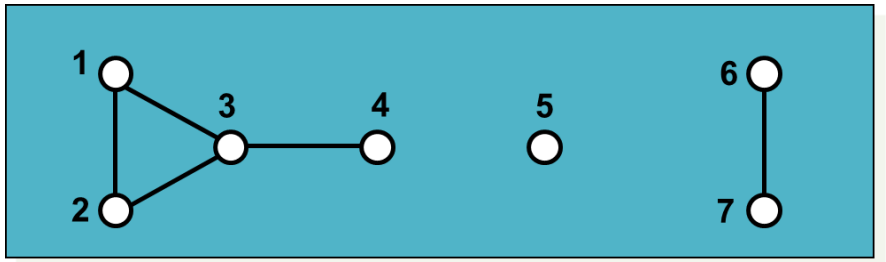
Procedimento Componente_conexo(G = (V,E))

```
para cada v ∈ V
    gerar(v)
para cada (u,v) ∈ E
    se busca(u) ≠ busca(v)
        fundir(u,v)
```

Conjuntos iniciais	{1}	{2}	{3}	{4}	{5}	{6}	{7}
(1,2)	{1,2}		{3}	{4}	{5}	{6}	{7}
(1,3)	{1,2,3}			{4}	{5}	{6}	{7}
(2,3)	{1,2,3}			{4}	{5}	{6}	{7}
(3,4)	{1,2,3,4}				{5}	{6}	{7}
(6,7)	{1,2,3,4}				{5}	{6,7}	

Uma Aplicação em Grafos

Verificar componentes conexos



Procedimento mesma_componente(u,v)

se busca(u) = busca(v)

retorne(verdadeiro)

retorne(falso)

Conjuntos iniciais	{1}	{2}	{3}	{4}	{5}	{6}	{7}
(1,2)	{1,2}		{3}	{4}	{5}	{6}	{7}
(1,3)	{1,2,3}			{4}	{5}	{6}	{7}
(2,3)	{1,2,3}			{4}	{5}	{6}	{7}
(3,4)	{1,2,3,4}				{5}	{6}	{7}
(6,7)	{1,2,3,4}				{5}	{6,7}	

Conjuntos Disjuntos - Fundir

A operação fundir consome significativamente mais tempo que as operações gerar e buscar.

- **Implementação Ingênua** – Fundir(x,y)

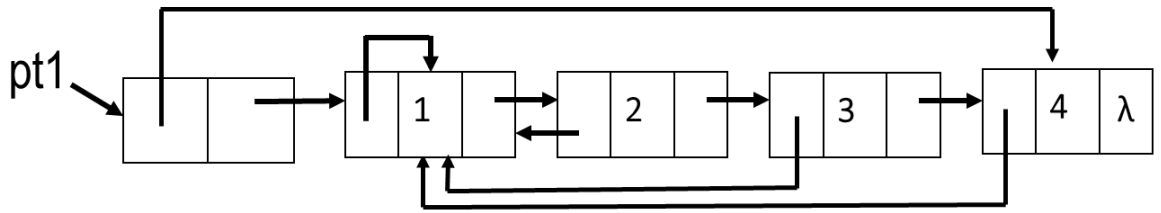
  Anexa lista y no final da lista x.

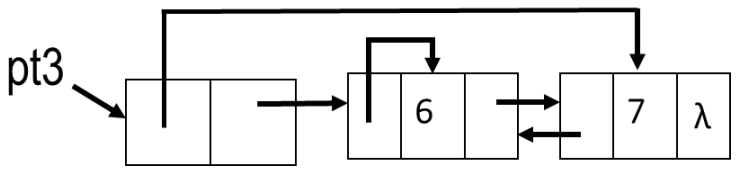
Ponteiro fim da lista x recebe ponteiro fim da lista y.

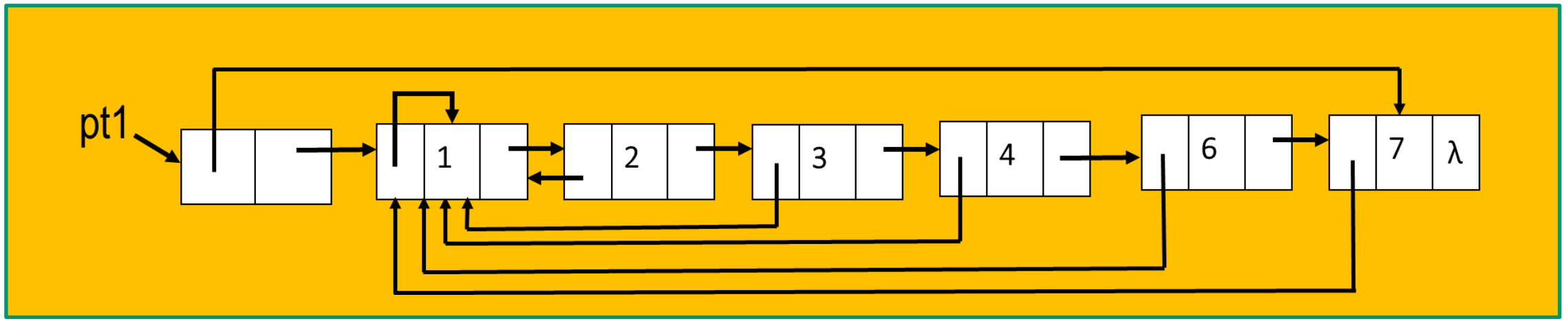
Cada elemento da lista y deve apontar para seu novo representante x.

Conjuntos Disjuntos - Fundir

- **Exemplo Implementação Ingênua** – Fundir(1,6)







Conjuntos Disjuntos - Fundir

Sequência de *m* operações gerar e fundir em *n* objetos pode requerer $\Theta(n^2)$ operações

- Supor objetos 1,...,*n*. Executar *n* operações gerar e *n*-1 operações fundir. Assim, *m* = 2*n*-1.
- Gastamos $\Theta(n)$ tempo para as *n* operações gerar ($\Theta(1)$ para cada uma).
- Supor que a *i*-ésima operação fundir opera *i* objetos (a lista maior é anexada ao final da menor), o número de objetos atualizados pelas *n*-1 operações fundir é

$$\sum_{i=1}^{n} i = \Theta(n^2)$$

Conjuntos Disjuntos - Fundir

Sequência de *m* operações gerar e fundir em *n* objetos pode requerer $\Theta(n^2)$ operações

- Análise Agregada

Como $m = 2n-1$ e necessitamos tempo $\Theta(n^2)$ na sequência de m operações, o tempo amortizado por operação é $\Theta(n)$.

Conjuntos Disjuntos – Fundir

- Suponha que no nó cabeça da lista guardamos, também, o tamanho da lista.

  Anexar a lista menor no final da lista maior.

  Uma operação fundir pode requerer $\Omega(n)$ se as duas listas tiverem tamanho $\Omega(n)$ (por exemplo, $n/2$) .

Conjuntos Disjuntos – Fundir

- Suponha que no nó cabeça da lista guardamos, também, o tamanho da lista.

  Anexar a lista menor no final da lista maior.

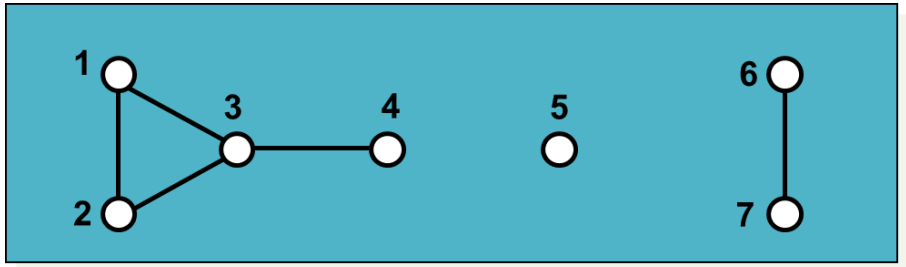
  Uma operação fundir pode requerer $\Omega(n)$ se as duas listas tiverem tamanho $\Omega(n)$ (por exemplo, $n/2$) .

- **Teorema: Usando esta regra, uma sequência de *m* operações gerar, buscar e fundir em *n* elementos, das quais *n* operações são gerar, requer tempo $O(m+n \log n)$.**

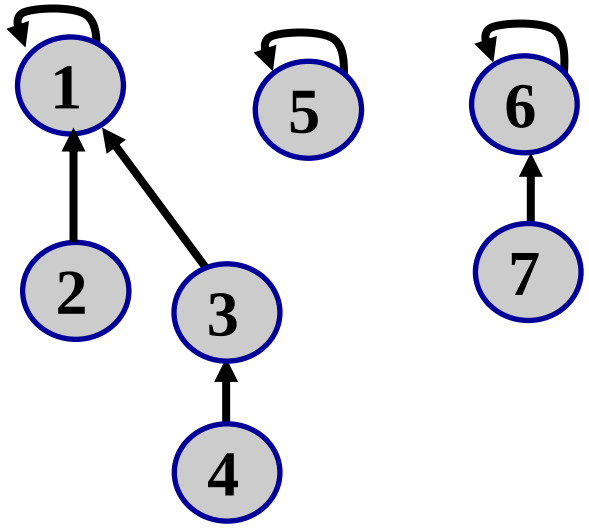
Prova. Cormen et al.

Floresta de Conjuntos Disjuntos

- Representar os conjuntos disjuntos por árvore enraizadas.



- Cada nó contém 1 elemento e cada árvore representa um conjunto.
- Cada elemento aponta para seu pai.
- A raiz de cada árvore contém o nó representante que aponta para si mesmo



Floresta de Conjuntos Disjuntos

Utilizar 2 heurísticas

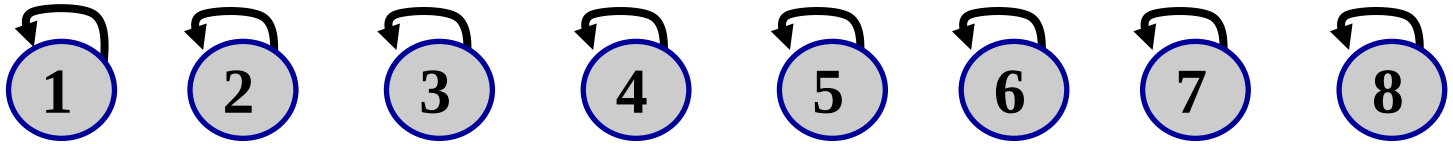
- União por posto
- Compressão de caminhos

Floresta de Conjuntos Disjuntos

- Gerar: cria uma árvore com 1 nó.

Exemplo: **Considerando *n* = 8**

Operação *Gerar*:



Floresta de Conjuntos Disjuntos

- Buscar: faz um caminho na árvore subindo pelos pais até encontrar o nó raiz.

- Fundir: faz o nó raiz de uma árvore apontar para a raiz da outra.



Fusão por Posto

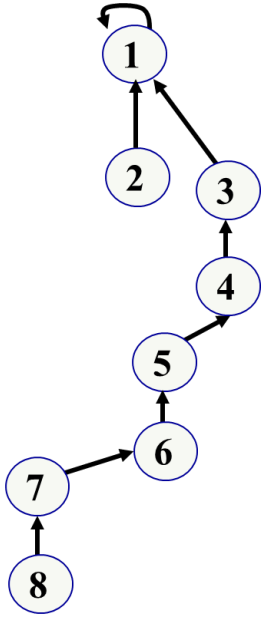
Ter uma ordem para cada árvore (limite superior para a altura da árvore)

- **Caso as árvores tenham tamanhos distintos, a raiz da árvore de menor ordem aponta para a raiz da árvore de maior ordem.**
- **Se os tamanhos forem iguais, escolhe-se uma das duas raízes para ser a raiz da nova árvore.**
- **- Se as duas árvores tiverem ordens iguais, a ordem da raiz da nova árvore é incrementada, caso contrário, não altera a ordem da raiz.**

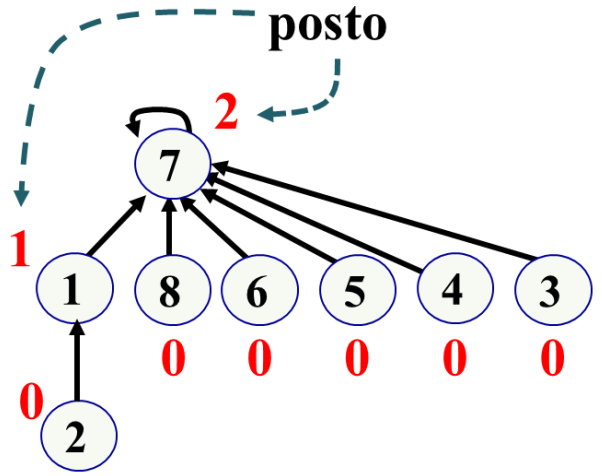
Fusão por Posto

- **fundir(1,2); fundir(7,8); fundir(6,7); fundir(5,6); fundir(4,5); fundir(3,4); fundir(1,3);**

- **Exemplo sem fusão por posto**

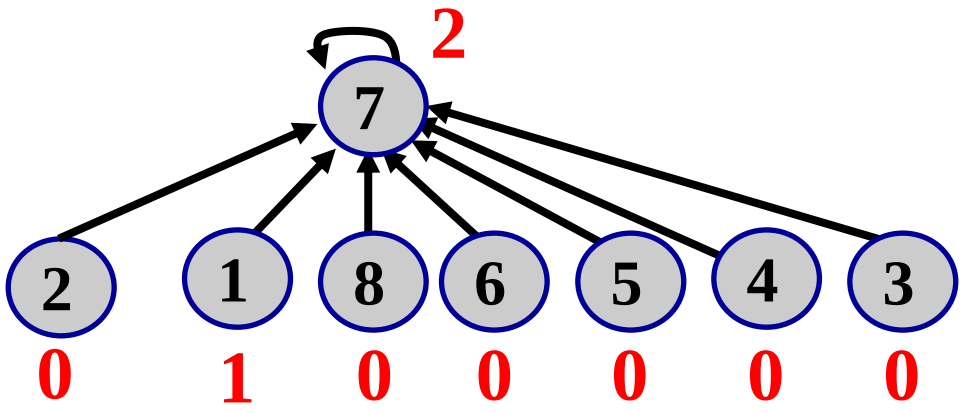


Exemplo com fusão por posto



Compressão de Caminhos

Fazer cada nó apontar para a raiz da árvore. (Não modifica os postos)



Implementação

Algoritmo gerar(x)

```
p[x] ← x
ordem[x] ← 0
```

Algoritmo busca(x)

```
se (x ≠ p[x])
    p[x] ← busca(p[x])
retorna(p[x])
```

Busca com compressão de caminhos

Implementação

```
Algoritmo fundir(x,y)
   unir(busca(x),busca(y))
```

```
Algoritmo unir(x,y)
   se ordem[x] > ordem[y]
      p[y] ← x
   senão
      p[x] ←y
      se (ordem[x] = ordem[y])
           ordem[y] ← ordem[y] + 1
```

Complexidade

Uma sequência de *m* operações *gerar*, fundir e buscar, das quais *n* operações são gerar, pode ser executada sobre uma floresta de conjuntos disjuntos com fusão por tamanho e compressão de caminhos em tempo de pior caso $O(m\alpha(n))$.

$\alpha(\ )$ é o inverso da função de Ackerman.

Prova. Cormen et al.

Função de Ackermann

$$A(i, j) = \begin{cases} 2j & i = j \\ 2 & j = 1 \\ A(i-1, A(i, j-1)) & \textit{caso contrário} \end{cases}$$

Da definição, tem-se que: $A(1,1) = 2$

$$A(1, j+1) = A(0, A(1, j)) = 2A(1, j) \quad \textit{para} \quad j \geq 1$$

Portanto, $A(1, j) = 2^j \quad \textit{para todo} \quad j$

$$A(2,1) = 2$$

$$A(2, j+1) = A(1, A(2, j)) = 2^{A(2, j)} \quad \textit{para} \quad j \geq 1$$

Função de Ackermann

Portanto,

$$A(2,1) = 1$$

$$A(2,2) = 2^{A(2,1)} = 2^2 = 4$$

$$A(2,3) = 2^{A(2,2)} = 2^{2^2} = 16$$

$$A(2,4) = 2^{A(2,3)} = 2^{2^{2^2}} = 65536$$

Em geral: $A(2, j) = \underbrace{2^{2^{\cdot^{\cdot^{\cdot^{2}}}}}}_{j}$

Função de Ackermann

A função de Ackermann cresce muito rapidamente.

A função $\alpha(i,j)$ é um tipo de inversa de A e é definida por:

$$\alpha(i,j) = \min\{k \mid k \geq 1 \text{ e } A(k, 4\lceil i/j \rceil) > \log j\}$$

A função α cresce muito vagarosamente.

Para valores muito grandes de j $\alpha(i,j) \leq 4$.

$\alpha(n,n)$ também é notado por $\alpha(n)$

FIM